



 Latest updates: <https://dl.acm.org/doi/10.1145/3783992>

SURVEY

A Survey of Operating System Kernel Fuzzing



PDF Download
3783992.pdf
18 December 2025
Total Citations: 0
Total Downloads: 110

Accepted: 01 December 2025

Revised: 27 October 2025

Received: 08 June 2025

[Citation in BibTeX format](#)

A Survey of Operating System Kernel Fuzzing

JIACHENG XU, Zhejiang University, China

HE SUN, Institute for Network Science and Cyberspace, Tsinghua University, China

SHIHAO JIANG, Zhejiang University, China

QINYING WANG*, Zhejiang University, China

MINGMING ZHANG, Zhongguancun Laboratory, China

XIANG LI, Nankai University, China

KAIWEN SHEN, Institute for Network Science and Cyberspace, Tsinghua University, China

CHARLES ZHANG, Tsinghua University, China

SHOULING JI, Zhejiang University, China

PENG CHENG*, Zhejiang University, China

JIMING CHEN, Zhejiang University, China

The Operating System (OS) kernel is foundational in modern computing, especially with the proliferation of diverse computing devices. However, its development comes with vulnerabilities that can lead to severe security breaches. Kernel fuzzing, a technique used to uncover these vulnerabilities, poses distinct challenges when compared to user-space fuzzing. These include the complexity of configuring the testing environment and addressing the statefulness inherent to both the kernel and the fuzzing process. Despite the significant interest from the community, a comprehensive understanding of kernel fuzzing remains lacking, hindering further progress in the field. In this paper, we present the first systematic study focused specifically on OS kernel fuzzing. We begin by outlining the unique challenges of kernel fuzzing, which distinguish it from those in user space. Following this, we summarize the progress of 107 academic studies from top-tier venues between 2017 and 2025. To structure this analysis, we introduce a stage-based fuzzing model and a novel fuzzing taxonomy that highlights nine core functionalities unique to kernel fuzzing. Each of these functionalities is examined in conjunction with the methodological approaches employed to address them. Finally, we identify remaining gaps in addressing challenges and outline promising directions to guide forthcoming research in kernel security.

CCS Concepts: • **Security and privacy** → **Operating systems security**; • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Fuzzing, Operating System Kernel

*These authors are co-corresponding authors.

Authors' Contact Information: Jiacheng Xu, stitch@zju.edu.cn, Zhejiang University, Hangzhou, China; He Sun, qldxtest@gmail.com, Institute for Network Science and Cyberspace, Tsinghua University, Beijing, China; Shihao Jiang, sh.jiang@zju.edu.cn, Zhejiang University, Hangzhou, China; Qinying Wang, wangqinying@zju.edu.cn, Zhejiang University, Hangzhou, China; Mingming Zhang, zhangmm@mail.zgclab.edu.cn, Zhongguancun Laboratory, Beijing, China; Xiang Li, lixiang@nankai.edu.cn, Nankai University, Tianjin, China; Kaiwen Shen, kaiwenshen17@gmail.com, Institute for Network Science and Cyberspace, Tsinghua University, Beijing, China; Charles Zhang, charles@vul337.team, Tsinghua University, Hangzhou, China; Shouling Ji, sji@zju.edu.cn, Zhejiang University, Hangzhou, China; Peng Cheng, lunarheart@zju.edu.cn, Zhejiang University, Hangzhou, China; Jiming Chen, cjm@zju.edu.cn, Zhejiang University, Hangzhou, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/12-ART

<https://doi.org/10.1145/3783992>

1 Introduction

OS kernels are central to modern computing systems, enabling communication between software and hardware components. Given the OS kernel's central role, its vulnerabilities lead to serious security breaches, including privilege escalation, sensitive data leakage, and remote code execution. For example, the *Dirty Cow* vulnerability [7] in the Linux kernel is infamous for enabling unauthorized privilege escalation, allowing attackers to manipulate and execute code at the root level. The risk posed by such vulnerabilities is amplified by growing and increasingly complex mobile environments, making it critical to secure against these threats. Meanwhile, fuzzing has proven to be an effective and practical approach for vulnerability discovery. Therefore, OS kernel fuzzing techniques have attracted significant attention from the research community [27, 28, 47, 119, 142].

Compared to user-space fuzzing, OS kernel fuzzing presents significant and complex challenges for the following reasons. First, unlike applications operating in controlled and uniform environments, kernel code interacts with a broad array of hardware components, each featuring its own drivers and peculiarities [104, 160]. This intricate interplay increases the risk of system-wide crashes or instability in the event of kernel faults, making it precarious [105, 112]. As a result, creating a consistent and reliable testing environment becomes particularly challenging. Second, synthesizing test cases for kernels is usually more challenging than for applications. The difficulty stems from the need to handle a wide variety of complex, structured inputs, e.g., system calls (syscalls) [52, 117, 142] and peripherals [40, 69], whose specifications are often deeply embedded within the kernel codebase. Finally, the kernel's inherent complexity and low-level nature introduce additional obstacles. It is challenging to precisely control kernel actions, monitor its internal state, and accurately interpret its responses to inputs during fuzzing [96]. These difficulties are further amplified by challenges related to scalability and lightweight design, which tend to become more pronounced as the fuzzing process grows or evolves.

Owing to these inherent complexities, OS kernel fuzzing techniques have become a major focus of extensive research. The rapidly expanding collection of OS kernel fuzzing techniques shows wide variation in goals and methods across different stages of the fuzzing pipeline. It is essential to conduct a deeper investigation into their shared characteristics and the specific challenges they aim to address. Additionally, assessing performance trade-offs and uncovering untapped opportunities for future advancements are vital to furthering progress in this field. However, thus far, no systematic review of the OS kernel has been conducted. Existing surveys mainly focus on general fuzzing techniques and evaluation criteria [80, 127, 157, 177]. Specifically, fuzzing for embedded systems, representing a related yet distinct line of work, is introduced [190].

To achieve this, we conduct an extensive review of 107 OS kernel fuzzing papers published in top-tier conferences between 2017 and August 2025, providing insights into the three research questions:

- **RQ1:** What desired functionalities distinguish OS kernel fuzzing from general fuzzing?
- **RQ2:** What are the techniques adopted for kernel fuzzing to satisfy the desired functionalities?
- **RQ3:** What are the challenges and future opportunities in the field of kernel fuzzing?

Although coverage- and crash-based metrics can provide insight into a kernel fuzzer's effectiveness, they are difficult to compare across different studies because of variations in definitions, experimental setups, and research objectives. Instead of relying solely on these metrics, we focus on the specific functionalities each technique implements, its applicability, and its methodological contributions. This functionality-oriented perspective offers a more practical understanding of a fuzzer's impact and utility. With this in mind, we first present the background of OS kernel fuzzing in **Section 2**, followed by our surveying methodology and results in **Section 3**. In **Section 4**, we outline the unique challenges of fuzzing in the OS kernel domain. In this regard, we propose a stage-based fuzzing model that decomposes the fuzzing process into discrete steps. For each stage, we describe the essential functionalities and key techniques employed by kernel fuzzers, addressing **RQ1**. Afterward, we provide a discussion of the existing proposals targeting each stage, i.e., *environment preparation* (**Section 5**), *input model* (**Section 6**) and *fuzzing loop* (**Section 7**). Drawing on qualitative assessment criteria, we emphasize

the implications learned from existing approaches and suggest promising technical solutions, responding to **RQ2**. Additionally, **Section 8** addresses the challenges and future directions of kernel fuzzing research, partially informed by a case study. This analysis contributes to answering **RQ3**. Finally, we provide a conclusion in **Section 9**.

In summary, we make the following key contributions:

- To the best of our knowledge, we present the first systematization of knowledge of OS kernel fuzzing techniques based on the review and analysis of 107 leading literature.
- We uncover the unique characteristics of OS kernel fuzzing compared to user-space fuzzing and develop a comprehensive taxonomy encompassing three core stages and nine essential functionalities.
- Using this taxonomy, we systematize advancements in the field and analyze gaps between desired functionalities and current practices.
- We identify existing challenges and highlight promising future directions.

2 Preliminaries

2.1 Role of OS Kernel

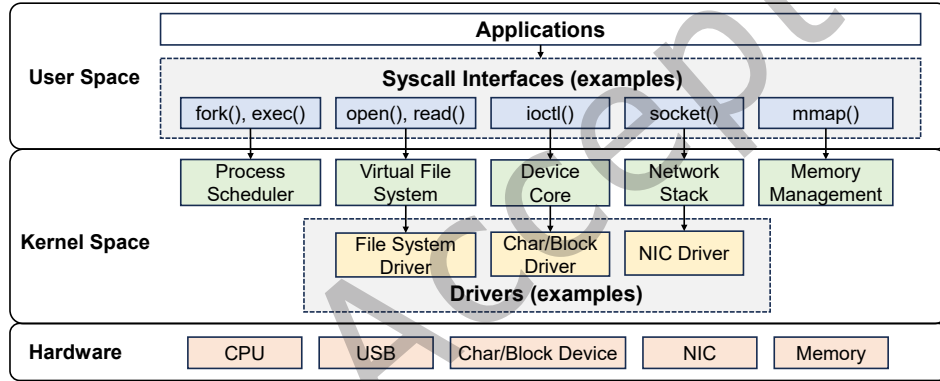


Fig. 1. Typical role and architecture of OS kernel in modern computer systems.

In modern computer systems, the OS kernel serves as the fundamental component responsible for managing hardware resources and providing essential abstractions to user-space applications. At the highest level, the user-space hosts applications and runtime libraries, which interact with the kernel through system calls (syscalls) to request various services. At the lowest level lies the hardware layer, comprising physical components such as the CPU, memory, and peripheral devices. Situated between these layers, the kernel operates with elevated privileges and functions as a critical mediator, coordinating communication between software and hardware. It performs core tasks such as process scheduling, memory management, file system operations, inter-process communication, and device control [50]. These functions ensure that multiple applications can execute concurrently, securely, and efficiently while abstracting away the complexity of direct hardware management. The typical role and position of the OS kernel within this layered architecture are illustrated in Figure 1. As the backbone of virtually all computing platforms—from traditional servers and desktops to mobile and embedded systems—the kernel constitutes a foundational software infrastructure upon which higher-level services and applications depend. Its reliability is therefore critical not only to system stability but also to performance and security. Therefore, kernels are subject to rigorous testing aimed at improving overall reliability. Among testing methods, fuzzing has

emerged as one of the most widely adopted approaches. For example, syzbot [152], which is integrated into the upstream Linux development process, has reported over 5,000 bugs.

2.2 Categories of OS Kernels

With the rapid expansion of computing devices, their scope has extended beyond traditional PCs and servers to encompass a broad spectrum of mobile terminals, embedded systems, and Internet-of-Things (IoT) environments. This diversification has led to a corresponding increase in the variety of OS kernels tailored to different application domains. General-purpose kernels are designed to provide comprehensive functionality and flexibility across multiple platforms, powering millions of devices in diverse domains, particularly within desktop environments. Prominent examples include Linux [151], Windows NT [33], and XNU [62], all of which emphasize user experience and broad compatibility. In contrast, real-time operating system (RTOS) kernels, such as FreeRTOS [2], prioritize deterministic scheduling and minimal latency to satisfy strict timing requirements in embedded and IoT settings. These kernels are often tightly coupled with heterogeneous hardware, presenting unique challenges for emulation and testing. Complementing these, mobile OS kernels, exemplified by the Android kernel, share certain traits with RTOS. At the same time, they are specifically optimized for energy efficiency, fine-grained access control, and the delivery of rich system services. These characteristics, while enhancing functionality, also introduce additional security risks through expanded interfaces. Beyond these categories, there are domain-specific kernels designed for resource-constrained IoT devices [11, 12, 86] as well as for confidential computing [3, 22]. The pervasive and foundational nature of OS kernels highlight the critical importance of ensuring their security, reliability, and resilience.

3 Paper Collection

3.1 Survey Methodology

We first outline the methodology we follow to collect papers. In this paper, the scope of OS kernels encompasses architectures that provide essential services necessary for system functionality, including hardware abstraction layer, driver model, memory management and scheduling [50]. We find that these kernels have evolved beyond traditional server or desktop models, taking on more diverse and specialized forms. Thus, our analysis includes kernels from general-purpose OS and their customizations [33, 46, 58, 62, 151], RTOS [1, 17, 108], TEE-OS [3, 36], ROS [15, 77, 131] and nano ones [2, 13], covering a range of environments from desktop to IoT devices. To ensure a comprehensive survey, we followed these steps:

- (1) **Venue selection.** Our study primarily focuses on papers published between 2017 and 2025 at *A** software engineering, cyber security and computer systems venues ranked by CORE2023 [61]. The selected publication venues are listed in Table 1.

Table 1. The selected top-tier venues from software engineering, cyber security and computer systems.

Research Topic	Type	Publications
Software Engineering	Conferences	ICSE, ISSTA, FSE, ASE
	Journals	TOSEM, TSE
Cyber Security	Conferences	USENIX Security, S&P, CCS, NDSS
	Journals	TDSC, TIFS
Computer System	Conferences	OSDI, SOSP, USENIX ATC, EuroSys, ASPLOS
	Journals	-

- (2) **Keyword match.** We then conducted a preliminary review of the literature and identified a set of keywords relevant to OS kernel fuzzing. These keywords were used to construct the final search query: (“operating system” OR “OS kernel” OR “Android services” OR “rtos” OR “firmware”) AND (“fuzzing” OR “fuzzer” OR “testing”). The search was performed using DBLP, which is a widely recognized bibliographic database in the field of computer science, and the results were restricted to papers published in the previously selected venues. This process resulted in an initial collection of 134 papers.
- (3) **Inclusion / exclusion criteria.** We manually inspected the papers collected in the previous step and retained those that met our inclusion criteria. Specifically, a paper was included in the survey if it proposed a novel fuzzing method explicitly designed for OS kernel or its subsystems. In contrast, studies that focused solely on bare-metal firmware or addressed fuzzing techniques not specific to kernels, such as those targeting general network protocols, were excluded. After applying these criteria, a total of 99 papers were selected for further analysis.
- (4) **Snowballing.** Finally, we performed both forward and backward snowballing to ensure broader coverage and capture relevant studies that may have been missed in the initial search. As a result, a total of 107 papers were identified for inclusion in the survey.

3.2 Survey Result

At the time of submission, our dataset included papers published between 2017 and August 2025, covering publicly available works (including early-access and preprint versions) from top-tier venues. Based on these collected papers, we analyze the current research landscape and trends in OS kernel fuzzing from three perspectives: publication venues and publication years. The statistical results are presented in Figure 2.

Publication venues. The surveyed literature is distributed across 20 different venues. A majority of OS kernel fuzzing papers are published in cyber security venues reflecting the high security risks associated with kernel vulnerabilities. This accounts for 67% of the total, as illustrated in Figure 2a. Software engineering venues follow, comprising 17%, while system conferences account for 9%. Notably, 28% of the papers were published at the USENIX Security Symposium, making it the most prominent venue in this research area.

Publication years. Figure 2b shows the distribution of papers by publication year. There is a clear upward trend in the number of kernel fuzzing publications, indicating the increasing importance of this research area and the growing attention it has received from the academic community. The apparent decline in 2025 is due to the fact that the year is not yet complete, and many papers have not been publicized at the time of this survey.

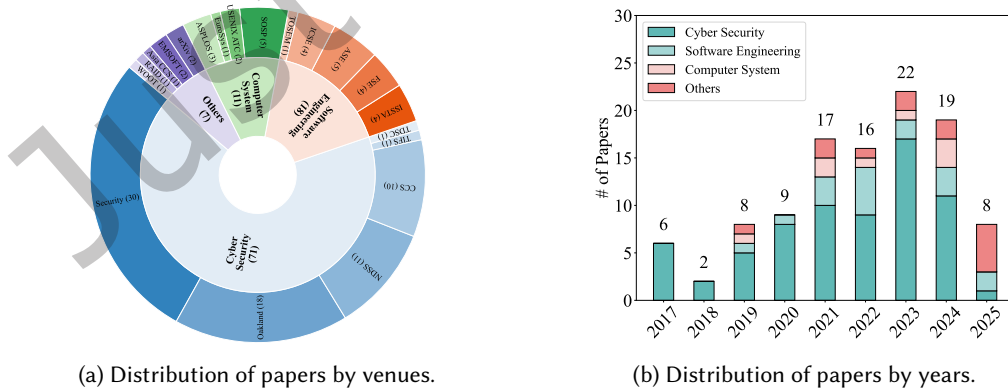


Fig. 2. Statistics of the collected papers.

4 Overview of Kernel Fuzzing

4.1 Challenges in Kernel Fuzzing

As previously discussed, the OS kernel occupies a unique position in the system architecture, bridging between the user space and the hardware layer. Thus, the complexity and privileged nature of OS kernels introduce distinct challenges in the design and implementation of fuzzing components, distinguishing kernel fuzzing from traditional fuzzing campaigns. To address **RQ1**, the challenges are categorized as follows, with a side-by-side comparison provided in Table 2:

- **C1: Constrained running environment.** In user-space, programs under test can typically run and restart in isolation with negligible overhead. By contrast, kernel fuzzing must operate in a constrained environment due to the tight coupling between OS kernels and underlying hardware. Kernels are fundamentally designed to support a broad spectrum of physical devices, and many of their functionalities depend on the availability of corresponding hardware dependencies, such as peripheral devices and embedded platforms [11, 31, 160]. Deploying real devices is costly and does not scale, while emulators offer only partial coverage of device diversity. For example, Linux supports more than 13,000 PCI devices, yet the widely used emulator [18] only implements fewer than 130 [184]. In addition, kernels are often deployed in diverse and challenging contexts, including closed-source systems and resource-constrained scenarios [22, 100, 189]. These conditions significantly complicate the collection of coverage information and hinder the deployment of advanced but heavyweight fuzzing techniques.
- **C2: Complex input interface.** User-space fuzzing usually focuses on fixed interfaces with well-documented inputs. However, the OS kernel inherently exposes more rich and highly complex interfaces for interacting with both user space and hardware, including syscalls, filesystems and peripheral I/O. Inputs for these interfaces are deeply structured and constrained by fine-grained semantics [75, 168]. For example, arguments for `ioctl` syscall vary widely across devices and often embed nested structures that must satisfy strict rules only encoded in millions of lines of code. Similarly, in subtasks such as directed kernel fuzzing (DKF), it often reveals the characteristics of discretized inputs [148]. Furthermore, vulnerabilities often manifest through cross-interface interactions, such as the combination of filesystem images with dependent syscalls [165]. Such an interplay between multiple interfaces significantly expands the input space and has no direct parallel in user-space fuzzing, posing substantial challenges in building accurate and comprehensive input models.
- **C3: High statefulness.** Statefulness exists in both user- and kernel-space programs, but it is far more pervasive and complex in kernels due to their global and long-lived execution context. Kernels often maintain and evolve internal states over time in response to prior interactions [164]. For example, as discussed later in Section 6.3, triggering a bug may depend on a specific sequence of syscalls `msync`, `mlockall` and `mmap`, reflecting implicit statefulness that is difficult to model. In this setting, branch coverage serves as a useful, though not definitive, fitness metric for greybox fuzzing. It may fail to capture the most informative feedback in kernel-specific scenarios [183]. Unlike protocol fuzzing, where states are often represented explicitly through finite-state machines (FSM), the definition and recognition of states within kernels remain largely unexplored. Additionally, the stateful nature of kernels introduces practical challenges: each reboot resets the internal state of the target system, forcing fuzzers to reinitiate the exploration process [140]. This leads to significant inefficiencies, as time and computational resources are repeatedly spent rediscovering previously explored states.

4.2 Key Functionalities

According to the survey results, we summarize that OS kernel fuzzing largely adheres to the standard methodology of typical fuzzing but is specifically adapted to address the aforementioned challenges. It involves generating

Table 2. Comparison of challenges in user-space and kernel fuzzing.

Challenges	User-space Fuzzing	Kernel Fuzzing	Examples
C1	minimal setup; ready-to-run programs	coupled with hardware; environment provisioning; resource-limited	hardware-in-the-loop for TEE [122]; hardware-based tracing [129]
C2	fixed, well-documented inputs	rich, deeply nested inputs; interplay across interfaces	spec. of ioctl buried in huge code space [52]; discretized inputs for DKF [148]; interplay of syscalls + filesystem images [167]
C3	often stateless or FSM-capturable	long-live, implicit states; hard to model; repeated re-exploration	syscall dependency (mlockall→msync) [53]; state-oriented fitness [183]

or mutating test cases that interact with kernel interfaces (e.g., syscalls) and executes them in a controlled environment [47]. During execution, the fuzzer utilizes runtime feedback (typically branch coverage) to steer execution toward unexplored space and increase the likelihood of discovering bugs, while continuously monitoring the target kernel for anomalous behavior such as crashes or hangs [135, 136]. As depicted in Figure 3, this workflow is typically structured around three core stages: environment preparation, input model and fuzzing loop, each playing a critical role in enabling effective fuzzing of OS kernels.

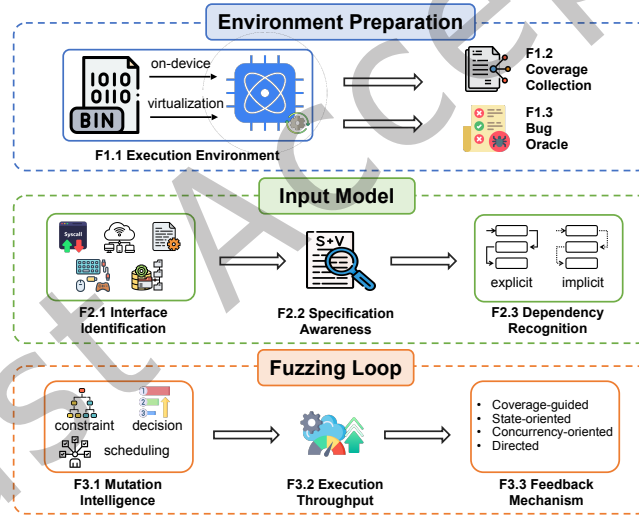


Fig. 3. Key stages and functionalities in kernel fuzzing.

To address the constrained environment challenge, the **environment preparation** establishes a stable and isolated execution context for the kernel, either through virtualization to enable scalable testing across multiple kernel instances [71, 106, 129], or on physical devices to ensure faithful representation of real-world behavior [109, 122, 139]. It also enables coverage tracking, and supports effective detection of potential vulnerabilities [88, 90, 138]. In this regard, three functionalities for developed environment preparation include: *execution environment* (F1.1), *coverage collection* (F1.2), and *bug oracle* (F1.3).

To handle the complexity of kernel input interfaces, an **input model** must first identify the relevant entry points through which the kernel is exercised (*interface identification*, *F2.1*), and then capture the structural and semantic rules governing these interfaces (*specification awareness*, *F2.2*), including expected data formats and constraints [25, 32, 59, 158]. Such a model further incorporates the key functionality of *dependency recognition* (*F2.3*), enabling fuzzers to generate test cases that are not only syntactically valid but also semantically meaningful.

Building on the prepared environment and the defined input model, the **fuzzing loop** continuously feeds generated or mutated test cases into the target kernel and leverages diverse feedback metrics specifically tailored to kernel fuzzing. In particular, to address the challenges posed by the kernel’s high degree of statefulness, the fuzzing loop aims to reduce the overhead of state re-initialization and improve execution throughput [72, 83, 140, 185]. Furthermore, it incorporates state-oriented feedback to strengthen the ability of state exploration [98, 122, 183]. Overall, a proficient fuzzing loop centers on three functionalities: *mutation intelligence* (*F3.1*), *execution throughput* (*F3.2*), and *feedback mechanism* (*F3.3*).

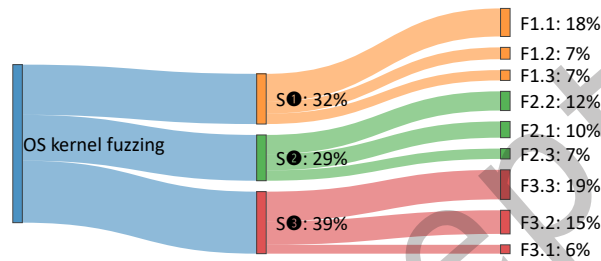


Fig. 4. Distribution of papers by target stages and functionalities.

We also study surveyed papers based on their primary targeted functionalities, with the results presented in Figure 4. In terms of core components, approximately 32% of the research concentrates on environment preparation for customized kernels. Additionally, 40% of the studies focus on optimizing the fuzzing loop, while 28% aim to improve the correctness and completeness of the input space. As for functionalities, environment preparation (18%) and feedback mechanisms (19%) emerge as prominent areas of interest, both representing aspects that require further research and practical advancement. The distribution of targeted functionalities also closely aligns with the key challenges previously identified (in Section 4.1) in OS kernel fuzzing. Furthermore, it is noteworthy that 59% of the papers specifically target Linux kernel fuzzing. This trend is largely driven by the availability of Syzkaller [47], which has established a mature and widely adopted infrastructure, thereby facilitating subsequent fuzzing research and optimization efforts.

Based on the above study, we answer **RQ1** as follows. We emphasize that **C1**, **C2** and **C3** distinguish OS kernel fuzzing from conventional fuzzing approaches. To address these challenges, we categorize nine key functionalities across three stages, among which *execution environment* (*F1.1*), *specification awareness* (*F2.2*) and *feedback mechanism* (*F3.3*) emerge as the most extensively studied aspects.

4.3 Key Techniques

In response to **RQ2**, we examine how existing techniques implement and advance the key functionalities identified above. Due to the aforementioned challenges, these studies have inspired a diverse range of technical innovations. These techniques can be organized into three categories, aligned with the stages of our model:

- **Environment-centric techniques.** These approaches focus on strengthening the fuzzing environment in terms of usability, scalability, and observability. To improve deployment, they use *on-device fuzzing*, which executes tests directly on the target hardware, and *rehosting-based fuzzing*, which migrates the execution

to an emulated environment (F1.1). For monitoring, they rely on *invasive instrumentation*, which modifies the target to collect fine-grained runtime data, and *non-invasive tracing*, which observes execution with minimal interference (F1.2). To expand bug detection, they employ *oracles* that identify not only *memory corruption* but also *non-crash bugs* (F1.3).

- **Input-centric techniques.** These methods focus on constructing and refining input models to improve fuzzing effectiveness. For test input, they employ *primary interface fuzzing*, which targets a single input channel (e.g., syscalls, peripherals, or filesystems), as well as *multiple-interface fuzzing*, which combines interfaces to uncover interaction behaviors (F2.1). To increase input validity, they utilize *static inference*, which extracts constraints from source code, and *dynamic analysis*, which derives constraints from runtime behaviors (F2.2). To capture stateful behaviors, they address *explicit dependencies*, where input relationships are explicitly defined, and *implicit dependencies*, where hidden semantic relations manifest through shared internal states (F2.3).
- **Loop-centric techniques.** These approaches aim to improve the fuzzing loop by enhancing execution efficiency and exploration focus. To accelerate execution, they adopt *virtualization enhancements*, which optimize VM performance, and *system snapshots*, which enable efficient restoration of kernel states (F3.1). To strengthen feedback, they apply *constraint solving* for precise path exploration, *thread scheduling* for concurrent execution control, and *decision intelligence* to prioritize promising test cases (F3.2). To guide exploration, they employ *directed fuzzing*, which steers inputs toward given targets, *state-oriented fitness*, which focuses on diverse program states, and *concurrency-oriented fitness*, which exposes thread interleavings systematically (F3.3).

Together, these techniques provide complementary solutions to improve efficiency, coverage, and bug-finding capability. In the following sections, we analyze each functionality in depth and derive a series of **implications** that summarize the strengths, limitations, and directions of these techniques. Each implication is grounded in the corresponding functionality discussed above (addressing RQ2) and extends the analysis toward unresolved challenges and future research opportunities (addressing RQ3).

5 Environment Preparation

5.1 Execution Environment

Two primary approaches provide the execution environment for OS kernel fuzzing: on-device fuzzing [37, 90, 122, 139] and emulation-based fuzzing [73, 105, 118, 123, 130, 140, 147]. We present a summary of the literature and the corresponding solutions employed within the environment preparation in Table 4.

5.1.1 On-device Fuzzing. An on-device fuzzer ensures the target kernel’s continuous and stable operation due to C1 by executing it on actual devices. It employs a user-space application or debugging system for coverage collection and bug detection, connecting directly to the kernel. This fuzzer effectively identifies defects related to unique hardware properties or configurations.

Local fuzzer. This type of fuzzer runs in the user space on a local machine and utilizes the exposed kernel interface to fuzz the kernel [128], as illustrated in Figure 5. It has limited ability to control and monitor the target kernel because of its low privilege. Even worse, it loses all execution information when the kernel crashes.

Remote fuzzer. The remote fuzzer connects to the target machine that is loaded with a kernel via serial ports [37, 90, 122, 139] or network [126]. This fuzzer requires a debugging system or probing module to be deployed in the target kernel, and the debug feature is utilized to control the target firmware. For instance, SyzTrust [122] and μ AFL [90] utilize ARM Coresight architecture to control the execution of embedded OSes, and PeriFuzz [139] designed their probing framework to manage the hardware boundary of a kernel.

Most of these approaches are open source and capable of supporting closed-source OSes. However, they often require intrusive control over OS execution. While the native execution environment of on-device fuzzers provides high stability and fidelity, it is limited in capacity and input execution speed when applied to RTOS on resource-constrained devices, such as ARM Cortex-M chips with clock speeds between 10 MHz and 600 MHz. Furthermore, on-device fuzzing is typically OS-specific and necessitates real, debug-enabled devices or elevated privileges for the fuzzer, which results in increased costs.

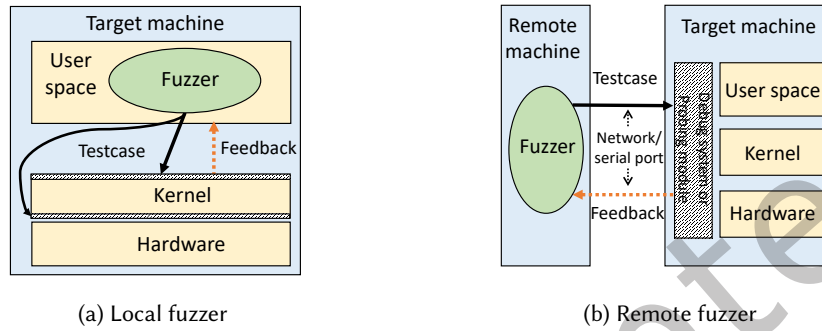


Fig. 5. Two typical on-device fuzzers.

5.1.2 Emulation-based Fuzzing. Loading the kernel into a virtual environment offers a more scalable approach with complete control for fuzzing OS kernels, providing a costless and effective solution for kernel introspection. However, the challenge lies in maintaining stability and fidelity to ensure that the emulated kernel operates consistently and without interruption. According to the previous work [39], there are two principal ways to construct the virtual environment: hardware emulation system and rehosted embedded system. Regarding the rehosted embedded system, we further categorize these rehosting techniques into hardware-in-the-loop, high-level emulation, MMIO modeling.

Full emulation-based fuzzing. Full hardware emulation replicates the functionalities of specific hardware accurately, allowing unmodified kernel execution and fuzzing when peripherals are adequately emulated [39]. Full emulation aims to implement as many peripherals as possible, providing relatively high stability and fidelity compared to rehosting. When the target emulator is open-source, developers have full control over both the emulator and the running kernel inside it, thereby maximizing the capability for introspection and analysis. Although there are three major emulators for OS kernels, including VMWare [153], VirtualBox [74], and QEMU [18], full emulation-based fuzzing predominantly utilizes QEMU. This preference is due to its effectiveness in handling general-purpose OS and embedded Linux environments. QEMU is particularly advantageous because it is open source and compatible with a range of fuzzing tools [47, 119, 129], making it the preferred choice. Regarding RTOS and TEE, QEMU offers only limited support. Consequently, existing fuzzers designed for these specialized kernels often rely on rehosting techniques. Since these kernels interact closely with the hardware or specialized architectures, achieving full emulation requires significant effort, making rehosting a more feasible approach for fuzzing in such environments. The speed of kernel operation and fuzzing depends on the machine running the emulator and whether any acceleration techniques are deployed. For instance, emulating a kernel on a machine with similar performance may be slower due to instruction translation overhead, while emulating low-performance kernels on high-performance machines can improve speed.

Rehosting-based fuzzing. While full emulation-based fuzzing is primarily designed for general-purpose OSes and their variants, rehosting-based fuzzing offers a complementary approach for RTOS such as Amazon

FreeRTOS [2], ARM Mbed [13], Zephyr [8], and LiteOS [24]. Unlike full emulation, a rehosted embedded system focuses on modeling only the essential features of target kernels required for fuzzing or dynamic analysis. This approach also provides full control over the target kernel and a comprehensive introspection. Based on our survey of state-of-the-art rehosting techniques, we identified three primary strategies: hardware-in-the-loop [82, 147], high-level hooking [31, 40, 69, 89], and MMIO modeling [23, 49, 54, 71, 187]. Hardware-in-the-loop, while useful, suffers from lower stability due to potential delays in forwarding hardware data, which can cause crashes during the operation and fuzzing of RTOS. Additionally, the speed bottleneck in this approach is tied to the execution speed of the hardware itself. The latter, MMIO modeling and high-level hooking, face significant challenges related to fidelity and stability, particularly in accurately simulating hardware behavior, such as DMA and interrupt emulation, and handling complex peripherals [108]. When fuzzing a kernel within a rehosted system, crashes may occur due to the absence of certain feature models. Similarly to full emulation, the speed of kernel operation and fuzzing depends on the machine running the emulator and used acceleration techniques.

Implication ①: OS-Agnostic Rehosting. Current kernel fuzzing environments struggle to balance stability, overhead, and introspection but face significant challenges in achieving OS-agnostic execution, particularly for RTOS and TEE. Fidelity remains a critical limitation, as fuzzers often fail or get stuck due to incomplete or inaccurate emulation. This highlights the need for more faithful rehosting techniques that can generalize across diverse OS platforms. Promising directions include lightweight full-system emulation tailored for specialized environments and improved rehosting frameworks that enhance stability and fidelity while minimizing engineering effort.

5.2 Coverage Collection

Coverage is a key indicator for evaluating the fuzzing effectiveness. To collect coverage, there are two principal ways: invasive instrumentation and non-invasive tracing.

Table 3. Coverage collection methodologies and their applications

Methodology	Technique	Literature	Use Case
Invasive Instrumentation	Compilers add instrumentation code to record coverage during compilation	[47] [139] [120] [52] [148] [66]	Applicable to kernels whose source code is available. It is mostly adopted for easy usage and high flexibility. The overhead is usually low.
	Emulators inject instrumentation code to trace coverage during dynamic binary translation	[105] [55] [31] [11] [149] [12]	Applicable to kernels that are binary-only but emulatable. The overhead is significantly higher than source-based instrumentation.
	Instrument a compiled kernel without altering its original functionality.	[170]	Applicable to binary-only kernels. The technique may compromise the kernel structure and thus is less adopted.
Non-invasive Tracing	Leverage built-in features of the kernel to trace coverage	[28] [158]	Depends on specific types of kernels.
	Leverage low-level hardware to collect coverage	[129] [122] [90]	Applicable to both source-based and binary kernels, especially on devices. It is architecture-specific and provides limited types of coverage.

5.2.1 Invasive Instrumentation. In essence, invasive instrumentation modifies target code during compilation or runtime, and exposes interfaces for tracking executed portions. Table 3 presents a detailed comparison of coverage collection techniques and their use cases.

Source-based instrumentation. Instrumenting the kernel during compilation is the most effective and intuitive method for coverage collection. For example, KCOV enhances fuzzing by injecting signals into basic

blocks, significantly improving bug discovery through code coverage [47, 66, 117, 138, 142, 154, 164]. However, this approach primarily targets bugs reachable via syscall inputs and has limitations in non-deterministic areas and non-syscall handlers. Solutions like PeriScope [139] and USBfuzz [120] have advanced coverage by focusing on fine-grained and remote collection methods, enabling more effective fuzzing in areas like driver operations and interrupts.

Dynamic instrumentation. When source code is unavailable, dynamic instrumentation provides an alternative for modifying a kernel without requiring access to its source. This approach typically involves an emulator that translates raw machine instructions into an intermediate representation. During this translation, instrumentation code is inserted to enable coverage collection [18]. A key prerequisite is that the kernel must be booted within the emulator. Hence, dynamic instrumentation is commonly integrated with emulation tools such as AFL-QEMU [178], enabling analysis of binary-only kernels for Linux [12, 31, 105, 149], RTOS [108], and TEE [55].

Binary rewriting. For binary only kernels, binary rewriting [35, 113, 182] provides a viable alternative. Adapting binary rewriting for kernel fuzzing presents additional hurdles, such as significant overhead that reduces fuzzing efficiency [106] and the complexity and ambiguities associated with static methods [35, 182]. Nonetheless, recent innovations [170] for macOS kernel extensions show promise, achieving cost-effective static binary rewriting by leveraging macOS's features for efficient coverage instruction injection.

5.2.2 Non-invasive Tracing. As a result of C1, instrumentation is often restricted because the kernel runs in privileged mode. The strong coupling with hardware leads to a lack of middleware support, thereby limiting observability. In such cases, fuzzers rely on limited interfaces, typically using debug checkpoints or hardware assistance.

Debug checkpoint. Similar to binary rewriting, tracing coverage through debug checkpoints involves setting checkpoints within the target and invoking them to gather feedback during execution. The key differences lie in their scalability and whether they leverage built-in kernel features. Tracing coverage via debug checkpoints is a target-specific approach that depends heavily on the target kernel's support. For example, SyzGen [28] uses macOS debugging tools to address challenges posed by closed-source kernels. Another example is on-device fuzzers that utilize hardware-based debugging, where feedback collection is closely tied to hardware features, such as embedded system debug units [37].

Hardware assistance. Hardware components like CPU have direct access to kernel space and every instruction, facilitating the acquisition of detailed feedback. Techniques like Intel PT and ARM ETM are widely used for capturing execution information, offering two key advantages. First, they ensure greater completeness and robustness in coverage collection. In contrast, user-space solutions are inherently limited, as they can only begin after the kernel or drivers have initialized and therefore miss the coverage generated during the bootstrap phase [53, 90, 184]. Second, these tools provide execution information from arbitrary OS code including closed-source ones, serving as OS-agnostic feedback mechanisms that support kernel fuzzing across multiple platforms [9, 14, 122, 129]. However, despite these hardware-based advantages, hardware-assisted approaches are limited to architecture-specific targets, similar to debug checkpoints.

Implication ②: Coverage Disparity. Source-based instrumentation has become the dominant approach in kernel fuzzing, adopted by approximately 77% of greybox fuzzers, owing to its flexibility and ability to collect fine-grained, customizable feedback (as detailed in Section 7.3). In contrast, binary-only kernels predominantly rely on OS- or architecture-specific techniques, 95% of which yield only coarse coverage metrics. This underscores the need for techniques that can deliver richer coverage feedback without requiring kernel source access.

Table 4. Summary of kernel fuzzers and their solutions used in environment preparation.

Year	Fuzzer	Func.	Type	Target	Env. Preparation		
					Exec. Env.	Coverage Collection	Bug Oracle
2017	Digtool [118]	1.1 1.3	●	Win.	CE	-	LA
2017	kAFL [129]	1.1 1.2	●	Win. & Linux	GPVM	HA	FS
2017	Syzkaller [47]	1.1 1.2 1.3	●	General	GPVM	SI	San. & FS & DD
2018	Charm [147]	1.1	●	Android	HIL	SI	San. & FS
2019	PeriScope [139]	1.1	●	Android	OD	SI	San.
2019	Hydra [78]	1.3	●	Linux	-	SI	San. & SC
2019	Unicorefuzz [105]	1.1 1.2	●	Linux	GPVM	DI	San.
2020	EASIER [121]	1.1 1.2	●	Android	Rehost	DI	FS
2020	HALucinator [31]	1.1	●	RTOS	CE	DI	DS
2020	PARTEMU [55]	1.1	●	TEE	CE	DI	FS
2020	USBFuzz [120]	1.1	●	Win. & Linux	Rehost	SI	San.
2021	Aafer et al. [10]	1.2	●	Android	OD	-	DT
2021	FirmGuide [101]	1.1	●	Emb. Linux	Rehost	-	FS
2021	ECMO [69]	1.1	●	Emb. Linux	Rehost	-	-
2021	DICE [108]	1.1	●	RTOS	CE	DI	San.
2022	μ AFL [90]	1.2	●	RTOS	HIL	HA	FS & TO
2022	EQUAFL [186]	1.1	●	Emb. Linux	CE	DI	FS & TO
2022	RoboFuzz [77]	1.3	●	ROS	HIL	SI	San. & DT & SC
2022	Tardis [133]	1.2	●	RTOS	GPVM	SI	-
2023	TEEzz [22]	1.1	●	TEE	OD	SI	FS
2023	FirmSolo [11]	1.1	●	Emb. Linux	CE	DI	FS
2023	KextFuzz [170]	1.2	●	MacOS	OD	BR	FS
2023	Greenhouse [149]	1.1	●	Emb. Linux	Rehost	DI	FS
2023	BoKASAN [29]	1.3	●	Win. & Linux	GPVM	-	DS
2024	BVF [144]	1.3	●	Linux	GPVM	SI	San.
2024	R2D2 [131]	1.2	●	ROS	GPVM	SI	San. & FS
2024	IPEA [137]	1.2 1.3	●	RTOS	HIL	SI	San.
2024	Monarch [103]	1.3	●	Linux	GPVM	SI	San. & SC
2024	SyzTrust [122]	1.2	●	TEE	HIL	HA	FS & TO
2024	Pandawan [12]	1.1	●	Emb. Linux	Rehost	DI	FS

1.1: The satisfied functionality of a method; ○ : Whitebox fuzzing; ● : Greybox fuzzing; ● : Blackbox fuzzing; Win: Windows; Emb. Linux: Embedded Linux; CE: Custom Emulation; GPVM: General-Purpose Virtual Machine; HIL: Hardware In the Loop; OD: On Device; HA: Hardware Assistance; SI: Source-based Instrumentation; DI: Dynamic Instrumentation; DT: Differential Testing; BR: Binary Rewriting; LA: Log Analysis; San: Sanitizer; FS: Fatal Signal; DD: Deadlock; SC: Semantic Checker; TO: Timeout; -: Not Specified.

5.3 Bug Oracle

Before starting fuzzing, it is necessary to design a bug oracle to detect bugs. Generally, these bug oracles target two primary types of issues: memory corruption and non-crash bugs. In addition, we discuss solutions for bug triage, focusing on strategies to effectively manage crashes.

5.3.1 Oracle for Memory Corruption. We categorize bug oracles for memory corruption into fatal signals and sanitizers.

Fatal Signal. Fatal signals are among the most widely used and intuitive bug oracles for detecting critical errors, such as illegal memory access. These signals manifest in various forms, including segmentation faults [11, 22, 121, 129], general protection faults [45, 47, 55], kernel panics [10, 101, 149, 170], and task hangs [186]. Additionally, kernel-level exception handling mechanisms [53, 122] are also used to indicate abnormal behavior. In on-device or hardware-in-the-loop testing scenarios, where direct inspection of execution output or heavyweight analysis is challenging, timeouts are often employed as a practical proxy [90, 122, 186]. However, relying solely on fatal signals is insufficient, as they may fail to detect logical or silent bugs that do not immediately disrupt system execution.

Sanitizer. Sanitizers have been the de facto oracles widely used by almost all kernel fuzzers. They detect bugs by instrumenting code and monitoring runtime behavior, with each type of bug requiring specific sanitizers. The research community has developed several kernel sanitizers, addressing vulnerabilities like use-after-free / out-of-bounds [4], data races [5] and undefined behaviors [6]. While effective, these sanitizers have limitations. First, they usually rely on source code instrumentation, which introduces significant overhead and is unsuitable for binary-only systems [35, 137]. Therefore, ongoing efforts aim to develop more efficient structures [64] and extend support to closed-source cases [29, 118]. For example, BoKASAN [29] leverages the Linux kernel's *ftrace* feature to insert hooks into critical functions, enabling dynamic instrumentation of binary-only kernels. Additionally, recent research [144] points out a fact that sanitizers do not cover the entire kernel, leaving many vulnerabilities undetected. It would be interesting to figure out how far these sanitizers are.

5.3.2 Oracle for Non-crash Bugs. Detecting silent bugs can be challenging since they do not always lead to a crash. It adds barriers to bug discovery. We summarize existing oracle for non-crash bugs into two types.

Differential testing. Differential testing addresses the challenge of detecting silent bugs by executing the same test case across multiple kernel versions or configurations. It is primarily designed to identify correctness issues, based on the assumption that the kernel should exhibit consistent behavior across different environments when given identical inputs. To support this approach, RoboFuzz [77] detects correctness bugs in the ROS by comparing execution results between simulators and real-world deployments. Similarly, physical discrepancies have been used as indicators of potential bugs in another case [10]. The primary challenge hindering the application of differential testing in kernels is the lack of suitable reference counterparts for comparison.

Semantic checkers. Traditional sanitizer usually target low-level memory errors like use-after-free or out-of-bounds accesses. By contrast, semantic bug detection focuses on detecting logic errors or high-level correctness, such as violations of properties and specifications [77, 78, 103]. It is used for ensuring that kernel behavior aligns with expected logical rules and operational semantics. For instance, in filesystem modules, semantic oracles verify if desired states or properties have been violated. Monarch's [103] detects semantic violations in memory, persistence, fault, and concurrency scenarios by comparing runtime states against symbolic execution based on POSIX specifications and distributed fault models. However, these checkers are often scenario-specific, limiting supported types of vulnerabilities and their general applicability.

5.3.3 Bug Triage. Beyond detecting crashes via bug oracles, kernel fuzzing critically relies on bug triage to manage the large volume of reports it produces. Triage typically consists of prioritization, minimization, and deduplication [107]. Compared with user space, prioritization in the kernel is substantially more complex. First, pervasive concurrency and statefulness make many bugs difficult to reproduce, rendering reproduction-based ranking unreliable [115]. Second, a report that appears "low-risk" may still expose severe consequences due to the diverse interactions within the kernel's large codebase [192]. To address these challenges, recent efforts focus on assessing potential impact through proximity-based search [94, 174, 192] or exploitability analysis [191]. Minimization in kernel fuzzing resembles that in user space. For example, Syzkaller reduces syscall sequences while preserving crash triggers [47, 164]. Deduplication inherits similar difficulties as prioritization. Besides, stacktrace-based heuristics popular in user space [102] are often insufficient for deduplication of kernel crashes,

since memory layout and thread interleavings cause the same underlying bug to manifest with subtly different behaviors [111]. Despite these advances, existing approaches primarily focus on memory corruption reports and thus heavily rely on information provided by sanitizers (e.g., stack traces and memory addresses). However, their effectiveness in detecting and analyzing non-crash bugs remains largely unexplored.

Implication ③: Beyond Memory Corruption. Enhancing kernel bug oracles requires a balance between detecting a variety of bug types, from memory corruption to more subtle non-crash issues. The detection of memory corruption vulnerabilities is relatively well-developed, with ongoing efforts primarily aimed at improving usability and minimizing overhead. In contrast, identifying semantic-aware bugs presents greater challenges, especially with the emergence of new classes of kernel vulnerabilities [81, 143]. Tackling non-crash bugs necessitates sophisticated approaches like differential testing and indicators for logical errors.

6 Input Model

Establishing the input model is a subsequent step after setting up a reliable testing environment. Blind fuzzing is inefficient for navigating the kernel's complex structure due to the vast input space. Therefore, a systematic approach to defining input synthesis is essential. In this section, we review studies on techniques for specifying desired input, emphasizing the importance of interface identification, specification awareness, and dependency recognition. Table 5 provides a summary of the literature and the corresponding solutions for the input model stage.

6.1 Interface Identification

The OS kernel, serving as the intermediary between user space and hardware, provides a myriad of interfaces. For a fuzzer to automatically and effectively detect vulnerabilities, it must first identify the interfaces that align with its objectives.

6.1.1 Primary Interfaces. There are primarily five types of fuzzing interfaces exposed by OS kernels, consisting of:

Syscall. As the primary interface between user space and kernel space, syscalls serve as the fundamental mechanism for invoking kernel-level functionality. In the latest Linux 6.14 release, the kernel exposes nearly 500 syscalls, not including the numerous variants that are controlled by command parameters (e.g., through the *cmd* argument). To capture this complexity, Syzkaller defines more than 8,000 syscall descriptions manually using its domain-specific language. These syscalls are vital for OS functionality and provide standardized interfaces for user space to perform a wide range of tasks. As such, they form the principal input surface for kernel fuzzing and are typically exercised in the form of syscall sequences. Given the extensive and evolving syscall interface, along with the limitations of manual specification, considerable research has been devoted to automating the extraction and analysis of syscall semantics and interfaces [32, 52, 96].

Peripheral devices. Peripheral devices communicate with the kernel through mechanisms such as MMIO [32, 40, 108, 134], Port I/O [120, 121, 147], DMA [104, 108, 139, 160], and interrupts [32, 134, 184]. These channels form a foundational interface for injecting test cases, facilitating the exploration of vulnerabilities within the OS-hardware communication layer. Thorough testing of these interfaces is critical, as they can serve as potential vectors for security breaches. Given the high cost and complexity of interacting with real hardware, practical approaches include device-based techniques [32, 109, 139, 146, 147] and device-free modeling [104, 134, 160, 184].

Filesystem. Filesystems are fundamental components of an OS kernel, crucial for managing user files and maintaining data consistency during system crashes. They are typically structured, complex binary blobs mounted as disk images. Users interact with a mounted filesystem image through a set of file operations (e.g., syscalls).

Some studies focus on mutating images as binary inputs [14, 129], while others limit themselves to generating operations [25, 47].

Network. Network access is a fundamental feature in modern OS kernels, including streamlined or minimalistic ones. It provides a practical interface for testing and analysis, especially in resource-constrained systems such as embedded devices and IoT, where other interfaces may be constrained or unavailable [26, 126].

Configuration. In highly configurable OS kernels, configuration plays a critical role in shaping the input space. For instance, the default configuration of Linux kernels excludes approximately 80% of code changes from the compiled binary [169]. Because configuration changes necessitate recompiling the kernel, configuration testing must strike a balance between bootability, code coverage, and build time [56].

6.1.2 Multi-dimensional Input. Traditional methods typically concentrate on a single primary interface. This approach, while effective, tends to overlook vulnerabilities within the interplay between various interfaces (C2). The need for a more holistic approach has become increasingly apparent [160]. As revealed by Janus [167], relying solely on one aspect of the filesystem—either the disk image or file operations—inevitably neglects the other. This results in incomplete and less effective testing. Therefore, Janus proposes a two-dimensional strategy that explores input space from both sides. Such an inherent synergy is not only applicable to syscall-filesystem but is also highly relevant in other scenarios, such as syscall-driver [63, 104, 121, 125, 171]. For example, driver fuzzing is another area where a multi-dimensional approach is necessary. Device drivers often interact with various kernel interfaces, including syscalls and peripheral interfaces. Recent work explores USB drivers through record-and-replay techniques [63] and host-gadget synergy [171], further highlighting the importance of considering cross-interface behavior. Given the existence of thousands of complex devices and drivers beyond USB, a general methodology to systematically capture and test the wide range of interface interactions in OS kernels is highly desired.

Implication ④: Interface Interactions. The diversity of kernel input interfaces underscores the complexity and breadth of input model in kernel fuzzing. Beyond primary interfaces, studies have shown using one-sided input alone can miss up to 77% of kernel code that would otherwise be explored through cross-interface interactions [167, 171]. Although several efforts have investigated specific cross-interface scenarios, these attempts remain fragmented and domain-specific. The gap underscores the need for fuzzing approaches that explicitly model and exercise interface interactions, rather than treating interfaces in isolation.

6.2 Specification Awareness

Recognizing specification requirements is critical for fuzzing as kernel interfaces expect inputs adhering to defined structures and formats. However, being grammar-aware while generating input is a complex task. First, kernels interact with various devices and software, leading to diverse input formats. Second, beyond syntax, understanding input semantics is also important, as syntactically correct inputs can vary greatly in their effects. Despite the emergence of specification free techniques [21], specification-based approaches remain the mainstream [22, 28, 34, 52, 96]. To address these challenges, researchers have focused on extracting grammar specifications as an integral component of input generation. These methods can generally be categorized into static and dynamic.

6.2.1 Static Inference. Before the advent of automation, developers typically relied on their domain expertise to manually craft syscall specifications [47]. This manual process was both labor-intensive and error-prone, significantly limiting the scalability of testing across a broader range of syscall targets. These challenges have driven the development of static specification inference techniques, which analyze kernel source code to infer expected input structures and identify constraints without requiring code execution [96, 145, 160, 184]. For

instance, DIFUZE [32] pioneered the use of heuristic static analysis to extract syscall specifications from kernel driver source code. Building upon this foundation, SyzDescribe [52] enhances both the depth and accuracy of inference by constructing a principled response model of kernel drivers. In certain cases, documentation can also support the analysis by providing supplementary information [30]. These rule-based approaches, while effective, often require continuous maintenance to accommodate evolving kernel targets and struggle with handling edge cases [168]. To address these limitations, LLM-based techniques have presented high potential. KernelGPT [168] leverages the code-understanding capabilities of LLMs to adaptively extract specifications from source code with minimal reliance on external knowledge. It achieves a $6\times$ increase in specification generation while maintaining 93.3% accuracy, substantially outperforming traditional methods. However, LLMs face their own challenges. Their difficulty in capturing implicit semantics arises because such semantics are not explicitly encoded in source code, but instead distributed across control flows, hidden in indirect function calls, or dependent on runtime conditions (e.g., device-specific logic in DRM subsystems). Current LLMs rely heavily on textual and structural cues from the training data, which makes it hard for them to infer these latent dependencies without additional program analysis support. Beyond static structure, recent study [145] highlights the influence of runtime parameters exposed via `sysfs`, underscoring the complexity of the specification inference. Despite the progress of automated approaches, specification generation for subsystems such as eBPF [60] and complex protocols [126] remains an open problem. These domains still depend heavily on manual analysis due to the scarcity of explicit specifications, the prevalence of implicit semantics, and the need for runtime context that neither static rules nor LLMs alone can fully resolve.

6.2.2 Dynamic Analysis. In contrast to static inference, dynamic approaches typically adopt an iterative strategy that leverages runtime information collected from executing targets to complement static analysis. Based on the method of data collection, existing dynamic techniques can be broadly categorized into two types: log analysis [10, 28, 51, 63] and dynamic probing [141, 170, 189]. Log analysis serves as an indirect strategy, particularly useful for binary-only kernels where direct program inspection is inconvenient. For example, targeting macOS, SyzGen [28] starts by generating initial syscall templates through trace analysis and subsequently refines argument types and constraints using symbolic execution. Similar log-based techniques have also been applied to analyze interactions in USB protocols [120] and the Android ecosystem [10]. Dynamic probing has become feasible due to the enhanced observability features integrated into modern operating system kernels. In Linux, eBPF can be repurposed to hook into syscall probes and access the corresponding file operations [141]. Similarly, macOS provides a kernel extension wrapper that offers comparable functionality and serves as an entry point for inference when combined with taint analysis [170]. Compared to static inference, dynamic approaches yield more accurate and realistic outputs. However, they may also suffer from false negatives, as certain drivers or components may not be active or accessible during runtime analysis.

Implication ⑤: Hybrid Generation. Specifications are crucial for grammar-aware fuzzing but remain difficult to generate due to diverse formats and implicit semantics. Static inference offers broad coverage but lacks adaptability. LLM-based methods improve generality by up to $6\times$ [168] but cannot handle indirect call cases, and dynamic analysis provides realism but overlooks inactive paths. These complementary strengths and weaknesses indicate that no single approach is sufficient. Hybrid solutions combining static, dynamic, and LLM-based techniques are needed to achieve comprehensive and accurate specification generation.

6.3 Dependency Recognition

One of the most critical characteristics of OS kernels is their statefulness, as discussed in C3. This nature necessitates a coordinated organization of test cases, referred to as explicit / implicit dependency.

Table 5. Summary of kernel fuzzers and their solutions used in input model.

Year	Fuzzer	Func.	Type	Target	Input Model		
					Interface Ident.	Spec. Awareness	Dependency Recognition
2017	IMF [51]	2.2 2.3	●	MacOS	Sys.	LA	Exp.
2017	Syzkaller [47]	2.2 2.3	●	General	Sys.	Manual	Exp.
2017	DIFUZE [32]	2.2	●	Linux & Android	Sys.	RA	-
2018	MoonShine [117]	2.3	○	Linux	Sys.	-	Exp. & Imp.
2019	Janus [167]	2.1	○	Linux	Sys. & FS	-	-
2020	Dogfood [25]	2.3	○	Linux	Sys.	-	Exp.
2020	EASIER [121]	2.1 2.2	○	Linux	Sys. & Periph.	DP	-
2020	FANS [96]	2.2 2.3	●	Android	Sys.	RA	Exp.
2020	SyzGen [28]	2.2 2.3	○	MacOS	Sys.	LA & SE	Exp.
2020	ACHyb [59]	2.3	○	Linux	Sys.	-	Exp.
2021	DYNAMO [34]	2.2	○	Android	Sys.	RA	-
2021	NTFUZZ [30]	2.2 2.3	●	Win.	Sys.	RA	Exp.
2021	HEALER [142]	2.3	○	Linux	Sys.	-	Exp. & Imp.
2021	PrintFuzz [104]	2.1 2.2	○	Linux	Sys. & Periph.	RA	-
2022	Dr. Fuzz [184]	2.2	○	Linux	Periph.	RA & SE	-
2022	KSG [141]	2.2	○	Linux	Sys.	DP	-
2023	Schiller et al. [126]	2.1 2.2	●	Android & RTOS	Network	Manual	-
2023	FuzzNG [21]	2.2	○	Linux	Sys.	Spec. Free	-
2023	SyzDescribe [52]	2.2	○	Linux	Sys.	RA	Exp.
2023	DEVFUZZ [160]	2.1	○	Win. & Linux	Sys. & Periph.	RA & SE	-
2023	TEEzz [22]	2.2 2.3	○	TEE	Sys.	RA	Exp.
2023	ReUSB [63]	2.1 2.2	○	Linux	Sys. & Periph.	LA	-
2023	KextFuzz [170]	2.2	○	MacOS	Sys.	DP	-
2023	BRF [60]	2.2	○	Linux	Sys.	Manual	-
2024	MOCK [164]	2.3	○	Linux	Sys.	-	Exp. & Imp.
2024	Countdown [16]	2.3	○	Linux	Sys.	-	Exp. & Imp.
2024	SyzGen++ [158]	2.2 2.3	○	Linux & MacOS	Sys.	LA & SE	Exp.
2024	SATURN [171]	2.1	○	Linux	Sys. & Periph.	-	-
2025	KernelGPT [168]	2.2	○	Linux	Sys.	LLMA	-
2025	Hasanov et al. [56]	2.1	○	Linux	Config.	-	-
2025	SyzParam [145]	2.2	○	Linux	Sys.	RA	-

1.1: The satisfied functionality of a method; ○ : Whitebox fuzzing, 1.1 : Greybox fuzzing, ● : Blackbox fuzzing;
Win: Windows; Sys: Syscall; FS: File System Image; Periph: Peripheral; Config: Configuration; LA: Log Analysis; RA: Rule-based Analysis; DP: Dynamic Probing; SE: Symbolic Execution; LLMA: LLM-based Analysis; Exp: Explicit; Imp: Implicit; -: Not Specified or Not Targeted.

6.3.1 Explicit Dependency. Explicit dependencies refer to the direct relationships where the output of one syscall directly influences the input of another, such as in resource assignment. In this context, syscalls that generate outputs are identified as producers, while those that consume these outputs are considered consumers. We define a syscall c_i as explicitly dependent on another syscall c_j when c_i is a consumer and c_j is a producer. If open is not executed or fails, subsequent syscalls like mmap cannot execute successfully. Beyond return values, syscalls can also accept parameters derived from other syscalls. Some studies have sought to identify explicit

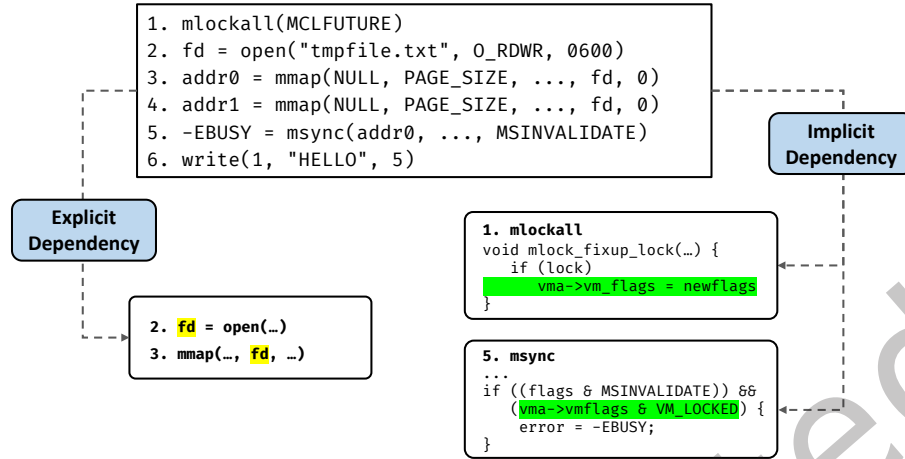


Fig. 6. Explicit and implicit dependencies between syscalls.

dependencies among syscalls through methods such as trace inference [28, 51, 158], layered model building [25] or producer-consumer analysis [47, 117, 142, 164]. For example, IMF [51] records the input and output values of hooked syscalls and then applies heuristic inference to logs, focusing on the order and value of entries. In the realm of producer-consumer analysis, fuzzers like Syzkaller perform type-based analysis on specifications, assigning higher priority to producer-consumer pairs.

6.3.2 Implicit Dependency. Implicit dependency, in contrast, is more subtle and mandates a sequence of syscalls without involving explicit producer-consumer relationships. It stems from the kernel's inherently stateful nature, characterized by extensive shared data structures and resources that can be accessed through multiple system calls. For instance, memory operations like `mlockall` and `msync` have no relevance in parameters or return value, but they operate on shared variables implicitly (shown in Figure 6), thereby creating implicit dependencies [117]. These dependencies are challenging to identify because they are often obscured within the vast and complex kernel codebase. Researchers currently use static analysis, dynamic analysis, and a combination of both to uncover these dependencies. Several studies have proposed static analysis techniques for identifying potential dependency pairs in kernel code, particularly when syscalls operate on shared global variables [41, 75, 117]. Although these methods can be informative, they are prone to generating false positives [66]. More recent research has advanced the field through dynamic dependency recognition, which offers more reliable results. For instance, HEALER [142] and MOCK [164] infer runtime dependencies by minimizing coverage and employ context-free and context-aware models, respectively. Furthermore, refcount has been explored as an additional means of representing implicit dependencies for mutation guidance [16]. Despite these advancements, the aforementioned methods are seldom applicable to closed-source targets. In fact, heuristic-based approaches remain dominant due to their practicality and usability.

Implication ⑥: Dependency Integration. Prior works have explored capturing or modeling explicit and implicit dependencies, yet no consensus has been reached among researchers. Ideally, these dependencies should be incorporated into specifications like *Syzlang*, which currently lacks support for them. Extending *Syzlang* to integrate these dependencies presents a promising direction.

7 Fuzzing Loop

Once the environment is configured and inputs defined, fuzzers initiate the fuzzing process. Traditional methods face kernel-specific challenges such as statefulness and concurrency. Functionalities required at this stage include execution throughput, mutation intelligence, and feedback mechanisms. Table 6 provides a summary of the literature and the corresponding solutions employed within the fuzzing loop.

7.1 Execution Throughput

Execution speed has a significant impact on performance of both user- and kernel-space fuzzing. However, the emulation-based architectures commonly employed in kernel fuzzing, together with its statefulness, necessitate solutions that differ substantially from those used in user space. Note that the techniques discussed here are developed based on their native environments and do not alter execution functionality, distinguishing them from those in *F1.1*.

7.1.1 Virtualization Enhancement. The introduction of a VM layer in kernel fuzzing enables improvements in execution throughput through enhanced virtualization efficiency.

Accelerated virtualization. Virtualization acceleration techniques [173] have been widely studied in the community. Existing fuzzing methods enable high performance virtualization through hardware assistance [129] and user-mode emulation [185]. However, these approaches are generally architecture- and fuzzer-specific, and thus limit their application.

Efficient synchronization. As the memory space of host and guest VMs is mutually isolated, their communication incurs significant overheads. For example, Syzkaller runs the fuzzer and executor inside the VM and synchronizes the state via RPC. Subsequent works mitigate the problem by proposing more efficient synchronization mechanisms, such as shared memory [83, 99, 142] and data transfer [99].

7.1.2 System Snapshot. The accumulated internal states may corrupt the kernel or interfere with subsequent executions. Hence, it is time-consuming but necessary to reboot the system regularly. The snapshot techniques save time and increase throughput by taking proper system snapshots and restoring them when necessary. The typical practice is to fork an initialized VM as a new instance [47, 119].

Lightweight snapshot. The native QEMU snapshot dumps all the CPU registers and the memory space and thus poses into files. Nevertheless, such a faithful snapshot may pose non-negligible overheads. A lightweight snapshot tailored to fuzzing is heavily desired. To achieve this, existing methods selectively restore memory pages on a Copy-on-Write principle [185], or customize the snapshot function upon QEMU/KVM for fuzzing adaptation [21, 43, 128]. Since this process operates at the emulation level, it can benefit multiple OS kernels that are virtualizable.

Checkpoint policy. It is typical of fuzzers to take a startup snapshot and restore it when necessary. However, the input executions undergo several similar phases besides startup. Hence, by properly creating continuous checkpoints [72, 140, 174], fuzzers can skip repeated steps and have direct access to the state that is established by time-consuming operations.

Implication ⑦: Throughput Gain. Studies have shown that secondary operations, such as data transfer, can account for up to 54% of fuzzing time [83, 99], significantly degrading overall performance. Consequently, optimizing virtualization for enhanced kernel fuzzer interaction and improving the bootstrap process for rapid recovery is key to increasing throughput. Future directions should aim at developing more universally applicable virtualization enhancements, creating lightweight snapshot techniques for fuzzing, and devising effective checkpoint policies to minimize redundant operations.

7.2 Mutation Intelligence

Although the input model reduces the search space, blind fuzzing still struggles to find bugs due to the complexity and micro-level variations in test cases. To address this, existing strategies are structured around three key phases: constraint solving, thread scheduling and decision intelligence.

7.2.1 Constraint Solving. While random fuzzing excels under lenient conditions, it struggles with stringent branch constraints, such as magic bytes and checksums, requiring extensive efforts to meet specific conditions. Integrating symbolic execution [176] with fuzzing has significantly boosted the ability to tackle complex constraints in user-space fuzzing, a strategy equally beneficial for kernel fuzzing. Hybrid approaches combining symbolic execution and fuzzing have been applied in kernel environments for interface recovery and value inference [28, 53, 141, 183], although scaling these methods for real-time use in complex kernels presents challenges, including indirect control transfers and path explosion [75]. Solutions specifically designed for kernel fuzzing aim to overcome these obstacles through indirect control flow transformation [75] and selective strategies [14, 27]. Besides, we also call for we emphasize the need to enhance the accessibility of the field, particularly given the current lack of available dynamic constraint-solving solutions.

7.2.2 Thread Scheduling. Concurrency-related kernel vulnerabilities emerge from the inherent complexity and unpredictability of non-deterministic kernel scheduling [42, 116]. Detecting these vulnerabilities requires careful consideration of both test inputs and specific thread interleavings, making precise control over threads essential for identifying concurrency issues. Rather than modifying the kernel scheduler—a process that is both labor-intensive and risky—the prevalent alternative is to control threads using delay injection [70, 165, 174] or using hypervisor-level control [65, 66]. Introducing delays between threads provides a straightforward means to influence thread scheduling [70, 165, 174]. However, this approach offers limited control over complex thread interactions and often results in significant performance degradation due to the added latency. To perform preemption, the hypervisor-based approaches either utilize hardware breakpoints as scheduling points and trap the thread in infinite loops [66, 67], or suspend thread execution on vCPU [43, 44]. Despite their effectiveness in enforcing thread control, such methods typically demand extensive modifications to both the hypervisor and the kernel. Consequently, they also suffer from high overhead and limit their scalability and usability severely [163]. In addition to the aforementioned approaches, research regarding RTOS has explored exposing concurrency bugs through task priority manipulation [132]. More recently, SECT [163] introduced a novel direction for thread scheduling. Rather than relying on external mechanisms, SECT targets the scheduling source itself by implementing a custom scheduler. This scheduler leverages the Linux eBPF feature and thus enables lightweight and flexible thread control.

7.2.3 Decision Intelligence. To maximize bug discovery under limited computational resources, fuzzers must make intelligent decisions when selecting seeds and mutation operators for each iteration. This phase represents the area of greatest similarity between user-space and kernel-space fuzzing. It often involves the use of runtime feedback (e.g., branch coverage) and optimization techniques such as reinforcement learning [156, 175, 179], information entropy [19], simulated annealing [91, 180], and particle swarm optimization [102]. These strategies are employed by fuzzers to support a range of testing objectives, including coverage maximization [45, 154, 164], concurrency analysis [44], vulnerability pattern detection [85], and reachability exploration [92, 180]. For instance, SyzVegas [154] utilizes a reinforcement learning algorithm to prioritize seeds and mutation strategies that yield higher coverage rewards. Similarly, SyzRisk [85] intends to allocate more fuzzing energy to inputs with higher probabilities of matching known vulnerability patterns. Although the challenges of decision intelligence are similar in both user space and kernel space, algorithms for the kernel domain have not been studied as extensively as their user-space counterparts []. It would be an interesting topic to investigate the effectiveness of user-space

scheduling algorithms in the context of kernel fuzzing and explore novel strategies specifically tailored to unique characteristics of kernel environments.

Implication ⑧: Native-feature Intelligence. While existing strategies have made notable progress, their intrusive nature and high overhead have constrained their practical applicability. These limitations make them unsuitable as a robust foundation for addressing challenges such as constraint solving and thread scheduling. Recent studies suggest that leveraging native kernel features offers a promising direction (e.g., achieving 11.4× speed-up in thread scheduling [163]), toward developing lightweight, scalable solutions that enhance mutation intelligence. Realizing this potential will require closer collaboration between the community and researchers.

7.3 Feedback Mechanism

As discussed in Section 5.2, existing kernel fuzzing proposals have improved feedback acquisition using both invasive instrumentation and non-invasive tracing. Establishing an efficient feedback mechanism for OS kernel fuzzing requires defining clear testing goals and proper fitness metrics.

7.3.1 Testing Goals. Fuzzers discover vulnerabilities with primary testing objectives, expanding code coverage (coverage-guided fuzzing) and prioritizing specific code locations (directed fuzzing).

Coverage-guided fuzzing. Kernel fuzzers typically employ a coverage-centric strategy, aiming to maximize execution path diversity—a signal that has proven effective for uncovering bugs [155]. These fuzzers assess input quality using fitness metrics, with basic block coverage [32] and branch coverage [65, 136] being the most commonly used. Additionally, state- and concurrency-oriented metrics have been developed to address kernel-specific characteristics, which will be discussed in more detail later. Due to the inherent complexity of operating system kernels, effective fuzzing requires a multi-dimensional feedback mechanism that integrates control flow analysis, data flow tracking, state exploration, and concurrency probing.

Directed fuzzing. Directed greybox fuzzing (DGF) have exhibited potential in tasks like patch testing [20, 84] and crash reproduction [94, 192]. Unlike coverage-centric fuzzing, DGF prioritizes seeds closer to specific target points, either manually set or indicated by sanitizers. However, its adoption in the kernel has been limited due to unique challenges of kernels. As mentioned in C2, one of the challenges arises from the testing interface. The input of DGF in user space are continuous and can be arbitrarily mutated. In contrast, kernel-space DGF inputs consist of a discrete set of syscalls, from which a relevant subset that actually reaches the target location must be identified. To address the challenge, existing work conducts static analysis from target code locations to assess reachability. Some of these efforts focus on the specific parameter values required by syscalls [135, 148, 172], while others emphasize optimizations tailored for particular subsystems [92]. Due to the large-scale codebases, a further challenge specific to kernel fuzzing is the high computational overhead and accuracy degradation associated with distance calculations. For instance, the state-of-the-art DGF method in user space, AFLGo [20], takes more than 16 hours to calculate the static distance for gVisor [48] on average [92]. In response, recent fuzzers have introduced various mitigation strategies, including enhanced distance metrics [92, 174], key object filtering [94, 192], and improved exploration–exploitation balancing techniques [92, 180]. For example, G-Fuzz [92] proposes a lightweight, directed fuzzing framework for gVisor. Its primary contribution lies in a fine-grained and efficient mechanism that substantially reduces the cost of distance computation while improving the handling of indirect calls through more precise analysis. These efforts highlight a blueprint for integrating DGF with various security tasks, with further applications (e.g., impact and exploitation assessment [94, 192]) in kernel security remaining a less unexplored topic.

7.3.2 Diverse Fitness. Classic code coverage lacks sensitivity to complex kernel conditions such as statefulness and thread interleaving. Most fuzzers focus on diverse fitness metrics beyond classic code coverage to approximate

Table 6. Summary of kernel fuzzers and their solutions used in fuzzing loop.

Year	Fuzzer	Func.	Type	Target	Fuzzing Loop		
					Thpt.	Mutation Intel.	Feedback Mechanism
2017	CAB-Fuzz [79]	3.2	●	Win.	-	CE	-
2019	RedQueen [14]	3.2	●	Linux	-	ISC	Branch
2019	Razzer [66]	3.2	●	Linux	-	HC	Branch
2019	FIRM-AFL [185]	3.1	●	Emb. Linux	LS	-	Branch
2020	HFL [75]	3.2	●	Linux	-	CE	Branch
2020	Krace [165]	3.3	●	Linux	-	DI	Branch & Inst. Pair
2020	Agamotto [140]	3.1	●	Linux	DC	-	Branch
2021	SyzVegas [154]	3.2	●	Linux	-	SS & MS	Branch
2021	Snowboard [43]	3.1	●	Linux	Snap.	HC	Branch
2021	Rtkaller [132]	3.2	●	RTOS	-	PA	Branch
2022	SFuzz [27]	3.2	●	RTOS	-	CE & TA	Branch
2022	RoboFuzz [77]	3.3	●	ROS	-	-	Branch & Semantic
2022	Conzzer [70]	3.1 3.3	●	Linux	-	DI	Branch & Concur. Call Pair
2022	GREBE [94]	3.3	●	Linux	-	-	Distance
2022	StrawFuzz [180]	3.2 3.3	●	Android	-	SS & MS	Distance & Key Var.
2022	FuzzUSB [76]	3.3	●	Linux	-	-	Branch & FSM
2022	StateFuzz [183]	3.3	●	Linux	-	-	Branch & State Var.
2023	Snowcat [44]	3.2	●	Linux	-	SS	Branch
2023	Thunder. [83]	3.1	●	Linux	VE	-	Branch
2023	SyzDirect [148]	3.3	●	Linux	-	-	Distance
2023	PLA [124]	3.2	●	Linux	-	SS	Branch
2023	SegFuzz [67]	3.3	●	Linux	-	HC	Branch & Segment
2023	KLAUS [159]	3.2 3.3	●	Linux	-	-	Distance
2023	DDRace [174]	3.3	●	Linux	DC	DI	Distance & Inst. Pair
2023	ACTOR [41]	3.3	●	Linux	-	-	Branch & Action
2023	G-Fuzz [92]	3.3	●	gVisor & Linux	-	SS	Branch & Distance
2023	Horus [99]	3.1	●	Linux	VE	-	Branch
2024	SyzTrust [122]	3.3	●	TEE	-	-	Branch & State Var.
2024	KBinCov [98]	3.3	●	Linux	-	-	Branch & State Var.
2024	SyzRisk [85]	3.1	●	Linux	-	SS	Branch
2025	SECT [163]	3.2	●	Linux	-	eBPFS	Branch

●: The satisfied functionality of a method; ○: Whitebox fuzzing. ●: Greybox fuzzing. ●: Blackbox fuzzing; Win: Windows; Emb. Linux: Embedded Linux; LS: Lightweight Snapshot; DC: Dynamic Checkpoint; VE: Virtualization Enhancement; CE: Concolic Execution; ISC: Input-to-State Correspondence; HC: Hypervisor-level Control; SS: Seed Scheduling; MS: Mutation Scheduling; PA: Priority Assignment; TA: Taint Analysis; DI: Delay Injection; eBPFS: eBPF-based Scheduling; FSM: Finite State Machine; -: Not Specified or Not Targeted.

the kernel under test comprehensively. These metrics, like block or edge coverage, guide fuzzers towards desired aspects of the target kernel, including state, concurrency. In response to **RQ2**, Table 7 presents a detailed comparison of fitness metrics and their use cases.

State-oriented fitness. As previously noted in **C3**, the high degree of statefulness constitutes a significant challenge for OS kernel fuzzing. Kernel state encompasses the execution context, including occupied resources like

registers and variables, distinct from user-space programs. OS kernels retain their values over time, accumulating internal states. This stateful nature sets kernel fuzzing apart from application fuzzing, requiring specific states to trigger vulnerabilities [98, 183]. Effective fuzzers navigate this complexity, targeting diverse and deep states. While some works [22, 28, 117, 142, 164] have examined states indirectly, a systematic approach with state-oriented fitness is still required to explore the space. Based on the type of state modeling, existing work can be divided into state-machine-based and state-variable-based. The state-machine-based approaches aim to identify concrete state machines embedded within the kernel under test. For instance, USB gadget stacks commonly implement various finite state machines (FSMs). To leverage this structure, FuzzUSB [76] combines static analysis with symbolic execution to detect state transitions in USB drivers. Based on these transitions, it infers potential state machines and retains inputs that trigger previously unexplored states. However, the state-machine-based approaches typically impose strict structural assumptions on the target system, and in many cases, explicit FSMs may not be present. In contrast, state-variable-based approaches offer a more practical alternative, as they do not rely on the existence of well-defined FSMs and can adapt to a broader range of system behaviors. The central idea behind this approach is to approximate kernel states using critical variables and monitor their value changes as an indicator of state coverage [98, 122, 183]. For example, StateFuzz [183] employs static analysis to identify candidate state variables, focusing specifically on those that can be traced back to global variables and are accessed by multiple operations. Unlike traditional coverage-guided fuzzers, StateFuzz retains an input if it triggers a new value range or an extreme value for a recognized state variable. Similarly, SyzTrust [122] identifies relevant state variables by summarizing critical structures associated with TEEs. Although these methods have shown success, their reliance on static analysis or heuristic modeling can lead to false positives and compromise state integrity. Open questions remain regarding the efficiency and soundness of state approximation.

Table 7. Diverse feedback fitness and its applications

Methodology	Technique	Literature	Use Case
State-oriented Fitness	Use FSM to model kernel states	[76]	Applicable to modules with clear state machines
	Use state variables as an abstraction of kernel states	[183] [122] [98]	Applicable to targets where states should be approximated
Concurrency-oriented Fitness	Function-level	[70]	Coarse-grained metric
	Instruction-level	[165]	High granularity but limited semantic metric
	Segment-level	[67]	Between function-level and instruction-level

Concurrency-oriented fitness. The widespread use of parallelization in OS kernels leads to a rise in concurrency bugs, like data races and deadlocks. Detecting these issues remains particularly challenging due to their inherently non-deterministic nature. While some techniques have been proposed to make thread scheduling more controllable [43, 66, 163], they typically operate without feedback from execution and thus manage thread scheduling in a largely uninformed manner. Traditional code coverage metrics fail to capture unique behaviors resulting from thread interleavings. It is evident that concurrency-oriented fitness is highly desired to facilitate a systematic exploration.

To effectively characterize the concurrency space, existing studies have defined metrics across multiple levels of granularity, including function-level, instruction-level, and segment-level [67, 70, 165, 174]. The function-level metric represents the coarsest granularity for analyzing concurrency, with Conzzer [70] serving as a representative example. The core intuition behind Conzzer is that if a function $func_a$ executes concurrently with another function $func_b$, then both the caller and callee of $func_a$ are also likely to execute concurrently

with *func_b*. Based on this insight, Conzzer introduces a *concurrent call pair* metric to capture and describe combinations of potential concurrent functions. Differently, Krace [165] operates at the instruction level and proposes a fine-grained metric known as *alias coverage*. This metric captures the memory locations accessed by concurrently executed instructions. While it offers detailed insight into low-level interactions, it does not fully capture the semantic characteristics of concurrency bugs. Notably, both Conzzer and Krace primarily focus on kernel filesystems, and their effectiveness in scaling to the broader kernel space remains an open question. To bridge the gap between function-level and instruction-level analysis, SegFuzz [67] presents an intermediate metric called *segment*. SegFuzz defines *segment* as a group of instructions that access shared memory objects. It strikes a balance between granularity and semantic relevance, offering a more practical representation of concurrent behavior. Despite these advancements, fuzzers targeting concurrency-oriented feedback continue to face significant challenges, particularly in terms of limited flexibility and substantial performance overhead [163].

Implication ⑨: Multi-feedback Prioritization. Existing OS kernel fuzzers mainly optimize the feedback mechanism based on the characteristics of target components. Exploring more targeted fitness metrics to uncover specific types of vulnerabilities is a valuable direction. However, as the use of multiple fitness metrics increases, the prioritization of feedback in the context of multi-feedback fuzzing has not yet been thoroughly studied, despite being a critical factor influencing the testing efficiency [154, 164].

8 Challenges and Opportunities

We have outlined potential directions of existing techniques in the preceding sections. To answer **RQ3**, in this section, we present a more detailed exploration of future directions that could enhance specific aspects of the fuzzing process and further improve kernel security.

Interactive driver fuzzing. As emphasized in implication ④, the attack surface of kernel drivers arises from both user space and peripherals. User-space programs interact with drivers via the syscall interface, such as `ioctl`, while devices connect with drivers through the peripheral interface. Both interfaces significantly impact the functionality of the drivers. Prior work [53] has demonstrated that some dependencies cannot be resolved without efforts from both sides. Nevertheless, existing works primarily concentrate on either user space or peripheral interactions when testing drivers, often feeding inputs from a single source. While earlier bugs have been systematically mitigated [160], the intricate internal states arising from the interactions between these two interfaces have received comparatively little attention, resulting in numerous vulnerabilities remaining unresolved. Recent studies [63, 171] have taken one step forward in the direction, although their approaches are limited to specific and can not scale. A potential solution is to develop a chronological driver model that focuses on code affected by both interfaces and to create a dual-interface fuzzing framework that simultaneously analyzes interactions from user space and peripherals while monitoring state changes.

Harnessing scheduler for concurrency. Kernel concurrency vulnerabilities are inherently more challenging to uncover compared to sequential ones due to the unpredictable nature of kernel scheduling [42, 43]. Despite advancements brought by various fuzzing techniques in *F3.2* and *F3.3*, these methods often necessitate significant modifications on kernels or emulators. These invasive customizations significantly hamper scalability and impose substantial performance overhead [43, 65, 67]. Recently, the introduction of the `sched_ext` [95] feature has opened new avenues for addressing this issue. Originally designed to enable flexible and extensible scheduler logic, `sched_ext` allows developers to modify scheduling behavior using pluggable eBPF programs [68]. As discussed in the implication ⑧, by designing schedulers specifically tailored for concurrency exploration, it becomes possible to precisely control thread interleaving in a customizable manner. At the same time, this approach retains the performance benefits of native execution and ensures forward compatibility [163]. Further

investigation is needed to explore the combination of `sched_ext` with concurrency-oriented fitness to improve lightweightness and generality.

In-domain benchmark construction. An in-domain benchmark is essential for fair and accurate evaluation, particularly given the rapid growth of kernel fuzzing techniques. Inspired by benchmarks developed for application fuzzing [57, 110, 114], an effective benchmark for kernel fuzzing should possess the following attributes: (a) Diversity: The benchmark should encompass a wide variety of bugs distributed across different modules. (b) Verifiability: It should employ reliable and measurable metrics. (c) Evolvability: As the kernel continuously evolves with the introduction of new features, the benchmark must also adapt to reflect the kernel development. Yet, the evaluation of kernel fuzzers is complicated by additional factors, as noted in *F1.1* and *F2.1*. A practical starting point would involve creating a benchmark specifically for Linux kernel, which typically offers superior infrastructure and has a broader impact. One potential approach is to combine with syzbot [152]. It includes a wide range of real-world bug reports from various types and different modules. These bugs are also accompanied detailed patch history and status, facilitating efficient triage. Additionally, syzbot’s continuous nature inherently supports the benchmark’s evolvability, allowing it to stay aligned with the ongoing development of Linux kernel.

LLM integration. LLMs have shown significant potential across a wide range of tasks [38, 88, 150, 166, 188], owing to their ability to understand and generate both natural language and code. Recent studies have also shed light on LLM integration with fuzzing workflows [161, 162], including syscall specifications generation [168] and dependency modeling [181]. While solutions represent an important step toward LLM-assisted fuzzing, significant limitations remain. As revealed in implication ⑤, a key challenge arises from the prevalence of indirect calls in the kernel [97]. Current LLM-based approaches derive specifications by analyzing relevant source code snippets. However, indirect calls are difficult to resolve from source code alone, and such methods fail in cases where critical semantics are embedded in indirect invocations (e.g., in DRM subsystems). At the same time, purely static analysis techniques, although more effective at handling indirect calls, lack the ability to capture textual information. Addressing this issue requires a balanced integration of LLMs with program analysis techniques, rather than relying solely on LLMs.

Impact of Rust. After years of discussion and development, the integration of Rust into the Linux kernel has become a reality. Although Rust code currently comprises only a small fraction of the codebase, plans are already in place for incorporating more extensive Rusty components, especially in drivers [87]. In light of this growing adoption, it is important to assess the implications for existing kernel fuzzing infrastructure and strategies. One key research direction involves examining the extent to which current frameworks effectively support the discovery of vulnerabilities in Rusty kernel code. For instance, the kernel coverage tool KCOV was originally designed with GCC in mind, and its compatibility and accuracy when applied to Rust code warrant further investigation. Moreover, as the focus of vulnerabilities in Rusty kernel components shifts from traditional memory corruption issues to logic errors [93], new categories of bugs may emerge. This evolution underscores the need for enhanced oracles capable of detecting logic vulnerabilities, which remains an open area for future research.

9 Conclusion

In this work, we conduct a systematic study of 107 OS kernel fuzzing papers published between 2017 and August 2025 in top-tier venues. We propose a comprehensive taxonomy of OS kernel fuzzing by introducing a stage-based fuzzing model and defining the desired functionalities at each stage. Leveraging this taxonomy, we analyze how contemporary techniques implement these functionalities, examine the gaps in current approaches, and explore potential solutions. Furthermore, we identify critical challenges faced by existing OS kernel fuzzing methodologies and highlight promising future research directions.

10 Acknowledge

We are grateful to the editors and the anonymous reviewers for their thoughtful feedback and constructive guidance, which significantly improved the quality of this work. This research was partially supported by the National Science Foundation of China (NSFC) under Grant No. 62293511, No. U244120033, U24A20336, 62172243, 62402425 and 62402418, the China Postdoctoral Science Foundation under No. 2024M762829, the Zhejiang Provincial Natural Science Foundation under No. LD24F020002, the “Pioneer and Leading Goose” R&D Program of Zhejiang under No. 2025C02033 and 2025C01082, and the Zhejiang Provincial Priority-Funded Postdoctoral Research Project under No. ZJ2024001.

References

- [1] 1987. VxWorks. <https://www.windriver.com/products/vxworks>.
- [2] 2003. FreeRTOS. <https://www.freertos.org/>.
- [3] 2014. OP-TEE: a Trusted Execution Environment. https://github.com/OP-TEE/optee_os.
- [4] 2015. The Kernel Address Sanitizer (KASAN). <https://docs.kernel.org/dev-tools/kasan.html>.
- [5] 2015. The Kernel Concurrency Sanitizer (KCSAN). <https://docs.kernel.org/dev-tools/kcsan.html>.
- [6] 2015. The Undefined Behavior Sanitizer (UBSAN). <https://docs.kernel.org/dev-tools/ubsan.html>.
- [7] 2016. CVE-2016-5195. <https://nvd.nist.gov/vuln/detail/cve-2016-5195>.
- [8] 2016. Zephyr: a new generation, scalable, optimized, secure RTOS for multiple hardware architectures. <https://zephyrproject.org/>.
- [9] 2020. Bugs on the Windshield: Fuzzing the Windows Kernel. <https://research.checkpoint.com/2020/bugs-on-the-windshield-fuzzing-the-windows-kernel/>
- [10] Yousra Aafer, Wei You, Yi Sun, Yu Shi, Xiangyu Zhang, and Heng Yin. 2021. Android SmartTVs Vulnerability Discovery via Log-Guided Fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2759–2776. <https://www.usenix.org/conference/usenixsecurity21/presentation/aafer>
- [11] Ioannis Angelakopoulos, Gianluca Stringhini, and Manuel Egele. 2023. FirmSolo: Enabling dynamic analysis of binary Linux-based IoT kernel modules. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5021–5038. <https://www.usenix.org/conference/usenixsecurity23/presentation/angelakopoulos>
- [12] Ioannis Angelakopoulos, Gianluca Stringhini, and Manuel Egele. 2024. Pandawan: Quantifying Progress in Linux-based Firmware Rehosting. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 5859–5876. <https://www.usenix.org/conference/usenixsecurity24/presentation/angelakopoulos>
- [13] ARM. 2013. Mbed OS: a platform operating system designed for the internet of things. <https://github.com/ARMmbed/mbed-os>.
- [14] Cornelius Aschermann, Sergej Schumilo, Tim Blazytko, Robert Gawlik, and Thorsten Holz. 2019. REDQUEEN: Fuzzing with Input-to-State Correspondence. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/redqueen-fuzzing-with-input-to-state-correspondence/>
- [15] Jia-Ju Bai, Hao-Xuan Song, and Shi-Min Hu. 2024. Multi-Dimensional and Message-Guided Fuzzing for Robotic Programs in Robot Operating System. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 763–778. doi: 10.1145/3620665.3640425
- [16] Shuangpeng Bai, Zhechang Zhang, and Hong Hu. 2024. Countdown: Refcount-guided Fuzzing for Exposing Temporal Memory Errors in Linux Kernel. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (Salt Lake City, UT, USA) (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 1315–1329. doi: 10.1145/3658644.3690320
- [17] Michael Barabanov. 1997. A linux-based real-time operating system. (1997).
- [18] Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator. In *Proceedings of the Annual Conference on USENIX Annual Technical Conference (Anaheim, CA) (ATEC '05)*. USENIX Association, USA, 41.
- [19] Marcel Böhme, Valentin J. M. Manès, and Sang Kil Cha. 2020. Boosting Fuzzer Efficiency: An Information Theoretic Perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 678–689. doi: 10.1145/3368089.3409748
- [20] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 2329–2344. doi: 10.1145/3133956.3134020

- [21] Alexander Bulekov, Bandan Das, Stefan Hajnoczi, and Manuel Egele. 2023. No Grammar, No Problem: Towards Fuzzing the Linux Kernel without System-Call Descriptions. In *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/no-grammar-no-problem-towards-fuzzing-the-linux-kernel-without-system-call-descriptions/>
- [22] Marcel Busch, Aravind Machiry, Chad Spensky, Giovanni Vigna, Christopher Kruegel, and Mathias Payer. 2023. TEEzz: Fuzzing Trusted Applications on COTS Android Devices. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1204–1219. doi: 10.1109/SP46215.2023.10179302
- [23] Chen Cao, Le Guan, Jiang Ming, and Peng Liu. 2020. Device-agnostic firmware execution is possible: A concolic execution approach for peripheral emulation. In *Annual Computer Security Applications Conference*. 746–759.
- [24] Qing Cao, Tarek Abdelzaher, John Stankovic, and Tian He. 2008. The LiteOS Operating System: Towards Unix-Like Abstractions for Wireless Sensor Networks. In *2008 International Conference on Information Processing in Sensor Networks (ipsn 2008)*. 233–244. doi: 10.1109/IPSIN.2008.54
- [25] Dongjie Chen, Yanyan Jiang, Chang Xu, Xiaoxing Ma, and Jian Lu. 2020. Testing file system implementations on layered models. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (Seoul, South Korea) (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 1483–1495. doi: 10.1145/3377811.3380350
- [26] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, XiaoFeng Wang, Wing Cheong Lau, Menghan Sun, Ronghai Yang, and Kehuan Zhang. 2018. IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*. The Internet Society. https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_01A-1_Chen_paper.pdf
- [27] Libo Chen, Quanpu Cai, Zhenbang Ma, Yanhao Wang, Hong Hu, Minghang Shen, Yue Liu, Shanqing Guo, Haixin Duan, Kaida Jiang, and Zhi Xue. 2022. SFuzz: Slice-Based Fuzzing for Real-Time Operating Systems. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (Los Angeles, CA, USA) (CCS '22)*. Association for Computing Machinery, New York, NY, USA, 485–498. doi: 10.1145/3548606.3559367
- [28] Weiteng Chen, Yu Wang, Zheng Zhang, and Zhiyun Qian. 2021. SyzGen: Automated Generation of Syscall Specification of Closed-Source macOS Drivers. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, Republic of Korea) (CCS '21)*. Association for Computing Machinery, New York, NY, USA, 749–763. doi: 10.1145/3460120.3484564
- [29] Mingi Cho, Dohyeon An, Hoyong Jin, and Taekyoung Kwon. 2023. BoKASAN: Binary-only Kernel Address Sanitizer for Effective Kernel Fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 4985–5002. <https://www.usenix.org/conference/usenixsecurity23/presentation/cho>
- [30] Jaeseung Choi, Kangsu Kim, Daejin Lee, and Sang Kil Cha. 2021. NtFuzz: Enabling Type-Aware Kernel Fuzzing on Windows with Static Binary Analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*. 677–693. doi: 10.1109/SP40001.2021.00114
- [31] Abraham A Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, Giovanni Vigna, Saurabh Bagchi, and Mathias Payer. 2020. HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 1201–1218. <https://www.usenix.org/conference/usenixsecurity20/presentation/clements>
- [32] Jake Corina, Aravind Machiry, Christopher Salls, Yan Shoshitaishvili, Shuang Hao, Christopher Kruegel, and Giovanni Vigna. 2017. DIFUZE: Interface Aware Fuzzing for Kernel Drivers. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 2123–2138. doi: 10.1145/3133956.3134069
- [33] Microsoft Corporation. 1993. Windows Kernel. <https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/>.
- [34] Abdallah Dawoud and Sven Bugiel. 2021. Bringing Balance to the Force: Dynamic Analysis of the Android Application Framework. In *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21-25, 2021*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/bringing-balance-to-the-force-dynamic-analysis-of-the-android-application-framework/>
- [35] Sushant Dinesh, Nathan Burow, Dongyan Xu, and Mathias Payer. 2020. RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization. In *2020 IEEE Symposium on Security and Privacy (SP)*. 1497–1511. doi: 10.1109/SP40000.2020.00009
- [36] Taras A. Drozdovskyi and Oleksandr S. Moliavko. 2019. mTower: Trusted Execution Environment for MCU-based devices. *Journal of Open Source Software* 4, 40 (2019), 1494. doi: 10.21105/joss.01494
- [37] Max Eisele, Daniel Ebert, Christopher Huth, and Andreas Zeller. 2023. Fuzzing Embedded Systems using Debug Interfaces. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (Seattle, WA, USA) (ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1031–1042. doi: 10.1145/3597926.3598115
- [38] Jueon Eom, Seyeon Jeong, and Taekyoung Kwon. 2024. Fuzzing JavaScript Interpreters with Coverage-Guided Reinforcement Learning for LLM-Based Mutation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (Vienna, Austria) (ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1656–1668. doi: 10.1145/3650212.3680389
- [39] Andrew Fasano, Tiemoko Ballo, Marius Muench, Tim Leek, Alexander Bulekov, Brendan Dolan-Gavitt, Manuel Egele, Aurélien Francillon, Long Lu, Nick Gregory, et al. 2021. Sok: Enabling security analyses of embedded systems via rehosting. In *Proceedings of*

- the 2021 ACM Asia conference on computer and communications security. 687–701.
- [40] Bo Feng, Alejandro Mera, and Long Lu. 2020. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 1237–1254. <https://www.usenix.org/conference/usenixsecurity20/presentation/feng>
 - [41] Marius Fleischer, Dipanjan Das, Priyanka Bose, Weiheng Bai, Kangjie Lu, Mathias Payer, Christopher Kruegel, and Giovanni Vigna. 2023. ACTOR: Action-Guided Kernel Fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5003–5020. <https://www.usenix.org/conference/usenixsecurity23/presentation/fleischer>
 - [42] Pedro Fonseca, Rodrigo Rodrigues, and Björn B. Brandenburg. 2014. SKI: Exposing Kernel Concurrency Bugs through Systematic Schedule Exploration. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 415–431. <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/fonseca>
 - [43] Sishuai Gong, Deniz Altınbüken, Pedro Fonseca, and Petros Maniatis. 2021. Snowboard: Finding Kernel Concurrency Bugs through Systematic Inter-Thread Communication Analysis. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (Virtual Event, Germany) (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 66–83. doi: 10.1145/3477132.3483549
 - [44] Sishuai Gong, Dinglan Peng, Deniz Altınbüken, Pedro Fonseca, and Petros Maniatis. 2023. Snowcat: Efficient Kernel Concurrency Testing using a Learned Coverage Predictor. In *Proceedings of the 29th Symposium on Operating Systems Principles (Koblenz, Germany) (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 35–51. doi: 10.1145/3600006.3613148
 - [45] Sishuai Gong, Wang Rui, Deniz Altınbüken, Pedro Fonseca, and Petros Maniatis. 2025. Snowplow: Effective Kernel Fuzzing with a Learned White-box Test Mutator. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1124–1138. doi: 10.1145/3676641.3716019
 - [46] Google. 2008. Android. <https://www.android.com/>.
 - [47] Google. 2015. syzkaller: an Unsupervised Coverage-Guided Kernel Fuzzer. <https://github.com/google/syzkaller>.
 - [48] Google. 2022. gVisor. <https://github.com/google/gvisor>.
 - [49] Eric Gustafson, Marius Muench, Chad Spensky, Nilo Redini, Aravind Machiry, Yanick Fratantonio, Davide Balzarotti, Aurélien Francillon, Yung Ryn Choe, Christophe Kruegel, et al. 2019. Toward the analysis of embedded firmware through automated re-hosting. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*. 135–150.
 - [50] Oliver Hahm, Emmanuel Baccelli, Hauke Petersen, and Nicolas Tsiftes. 2016. Operating Systems for Low-End Devices in the Internet of Things: A Survey. *IEEE Internet of Things Journal* 3, 5 (2016), 720–734. doi: 10.1109/JIOT.2015.2505901
 - [51] HyungSeok Han and Sang Kil Cha. 2017. IMF: Inferred Model-Based Fuzzer. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 2345–2358. doi: 10.1145/3133956.3134103
 - [52] Yu Hao, Guoren Li, Xiaochen Zou, Weiteng Chen, Shitong Zhu, Zhiyun Qian, and Ardalan Amiri Sani. 2023. SyzDescribe: Principled, Automated, Static Generation of Syscall Descriptions for Kernel Drivers. In *2023 IEEE Symposium on Security and Privacy (SP)*. 3262–3278. doi: 10.1109/SP46215.2023.10179298
 - [53] Yu Hao, Hang Zhang, Guoren Li, Xingyun Du, Zhiyun Qian, and Ardalan Amiri Sani. 2022. Demystifying the Dependency Challenge in Kernel Fuzzing. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 659–671. doi: 10.1145/3510003.3510126
 - [54] Lee Harrison, Hayawardh Vijayakumar, Rohan Padhye, Koushik Sen, and Michael Grace. 2020. PARTEMU: Enabling Dynamic Analysis of Real-World TrustZone Software Using Emulation. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 789–806. <https://www.usenix.org/conference/usenixsecurity20/presentation/harrison>
 - [55] Lee Harrison, Hayawardh Vijayakumar, Rohan Padhye, Koushik Sen, and Michael Grace. 2020. PARTEMU: Enabling Dynamic Analysis of Real-World TrustZone Software Using Emulation. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 789–806. <https://www.usenix.org/conference/usenixsecurity20/presentation/harrison>
 - [56] Sanan Hasanov, Stefan Nagy, and Paul Gazzillo. 2025. A Little Goes a Long Way: Tuning Configuration Selection for Continuous Kernel Fuzzing. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 521–533. doi: 10.1109/ICSE55347.2025.00042
 - [57] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2022. Magma: A Ground-Truth Fuzzing Benchmark. *SIGMETRICS Perform. Eval. Rev.* 49, 1 (jun 2022), 81–82. doi: 10.1145/3543516.3456276
 - [58] Joachim Henkel. 2006. Selective revealing in open innovation processes: The case of embedded Linux. *Research Policy* 35, 7 (2006), 953–969. doi: 10.1016/j.respol.2006.04.010
 - [59] Yang Hu, Wenxi Wang, Casen Hunger, Riley Wood, Sarfraz Khurshid, and Mohit Tiwari. 2021. ACHyb: a hybrid analysis approach to detect kernel access control vulnerabilities (*ESEC/FSE 2021*). Association for Computing Machinery, New York, NY, USA, 316–327. doi: 10.1145/3468264.3468627
 - [60] Hsin-Wei Hung and Ardalan Amiri Sani. 2024. BRF: Fuzzing the eBPF Runtime. *Proc. ACM Softw. Eng.* 1, FSE, Article 52 (jul 2024), 20 pages. doi: 10.1145/3643778

- [61] ICORE. [n. d.]. CORE2023. <https://www.core.edu.au/conference-portal>.
- [62] Apple Inc. 2001. XNU Kernel (macOS). <https://opensource.apple.com/source/xnu/>.
- [63] Jisoo Jang, Minsuk Kang, and Dokyung Song. 2023. ReUSB: Replay-Guided USB Driver Fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 2921–2938. <https://www.usenix.org/conference/usenixsecurity23/presentation/jang>
- [64] Yuseok Jeon, WookHyun Han, Nathan Burow, and Mathias Payer. 2020. FuZZan: Efficient Sanitizer Metadata Design for Fuzzing. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 249–263. <https://www.usenix.org/conference/atc20/presentation/jeon>
- [65] Dae R. Jeong, Yewon Choi, Byoungyoung Lee, Insik Shin, and Youngjin Kwon. 2024. OZZ: Identifying Kernel Out-of-Order Concurrency Bugs with In-Vivo Memory Access Reordering. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 229–248. doi: 10.1145/3694715.3695944
- [66] Dae R. Jeong, Kyungtae Kim, Basavesh Shivakumar, Byoungyoung Lee, and Insik Shin. 2019. Razzer: Finding Kernel Race Bugs through Fuzzing. In *2019 IEEE Symposium on Security and Privacy (SP)*. 754–768. doi: 10.1109/SP.2019.00017
- [67] Dae R. Jeong, Byoungyoung Lee, Insik Shin, and Youngjin Kwon. 2023. SegFuzz: Segmentizing Thread Interleaving to Discover Kernel Concurrency Bugs through Fuzzing. In *2023 IEEE Symposium on Security and Privacy (SP)*. 2104–2121. doi: 10.1109/SP46215.2023.10179398
- [68] Jinghao Jia, YiFei Zhu, Dan Williams, Andrea Arcangeli, Claudio Canella, Hubertus Franke, Tobin Feldman-Fitzthum, Dimitrios Skarlatos, Daniel Gruss, and Tianyin Xu. 2023. Programmable System Call Security with eBPF. arXiv:2302.10366 [cs.OS] <https://arxiv.org/abs/2302.10366>
- [69] Muhui Jiang, Lin Ma, Yajin Zhou, Qiang Liu, Cen Zhang, Zhi Wang, Xiapu Luo, Lei Wu, and Kui Ren. 2021. ECMO: Peripheral Transplantation to Rehost Embedded Linux Kernels. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) (CCS '21). Association for Computing Machinery, New York, NY, USA, 734–748. doi: 10.1145/3460120.3484753
- [70] Zu-Ming Jiang, Jia-Ju Bai, Kangjie Lu, and Shi-Min Hu. 2022. Context-Sensitive and Directional Concurrency Fuzzing for Data-Race Detection. In *29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24-28, 2022*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/auto-draft-198/>
- [71] Evan Johnson, Maxwell Bland, YiFei Zhu, Joshua Mason, Stephen Checkoway, Stefan Savage, and Kirill Levchenko. 2021. Jetset: Targeted Firmware Rehosting for Embedded Systems. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 321–338. <https://www.usenix.org/conference/usenixsecurity21/presentation/johnson>
- [72] Joonkyo Jung, Jisoo Jang, Yongwan Jo, Jonas Vinck, Alexios Voulimeneas, Stijn Volckaert, and Dokyung Song. 2025. Moneta: Ex-Vivo GPU Driver Fuzzing by Recalling In-Vivo Execution States. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/moneta-ex-vivo-gpu-driver-fuzzing-by-recalling-in-vivo-execution-states/>
- [73] Sylvester Keil and Clemens Kolbitsch. 2007. Stateful fuzzing of wireless device drivers in an emulated environment. *Black Hat Japan* (2007).
- [74] Rida Khan, Nouf AlHarbi, Ghadi AlGhamdi, and Lamia Berriche. 2022. Virtualization software security: oracle VM VirtualBox. In *2022 Fifth International Conference of Women in Data Science at Prince Sultan University (WiDS PSU)*. IEEE, 58–60.
- [75] Kyungtae Kim, Dae R. Jeong, Chung Hwan Kim, Yeongjin Jang, Insik Shin, and Byoungyoung Lee. 2020. HFL: Hybrid Fuzzing on the Linux Kernel. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/hfl-hybrid-fuzzing-on-the-linux-kernel/>
- [76] Kyungtae Kim, Taegyu Kim, Ertza Warraich, Byoungyoung Lee, Kevin R. B. Butler, Antonio Bianchi, and Dave Jing Tian. 2022. FuzzUSB: Hybrid Stateful Fuzzing of USB Gadget Stacks. In *2022 IEEE Symposium on Security and Privacy (SP)*. 2212–2229. doi: 10.1109/SP46214.2022.9833593
- [77] Seulbae Kim and Taesoo Kim. 2022. RoboFuzz: fuzzing robotic systems over robot operating system (ROS) for finding correctness bugs. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) (ESEC/FSE 2022). Association for Computing Machinery, New York, NY, USA, 447–458. doi: 10.1145/3540250.3549164
- [78] Seulbae Kim, Meng Xu, Sanidhya Kashyap, Jungyeon Yoon, Wen Xu, and Taesoo Kim. 2019. Finding semantic bugs in file systems with an extensible fuzzing framework. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) (SOSP '19). Association for Computing Machinery, New York, NY, USA, 147–161. doi: 10.1145/3341301.3359662
- [79] Su Yong Kim, Sangho Lee, Insu Yun, Wen Xu, Byoungyoung Lee, Youngtae Yun, and Taesoo Kim. 2017. CAB-Fuzz: Practical Concolic Testing Techniques for COTS Operating Systems. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. USENIX Association, Santa Clara, CA, 689–701. <https://www.usenix.org/conference/atc17/technical-sessions/presentation/kim>
- [80] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) (CCS '18). Association for Computing Machinery, New York, NY, USA, 2123–2138. doi: 10.1145/3243734.3243804

- [81] Jakob Koschel. 2025. *Uncovering New Classes of Kernel Vulnerabilities*. PhD-Thesis - Research and graduation internal. Vrije Universiteit Amsterdam. doi: 10.5463/thesis.833
- [82] Karl Koscher, Tadayoshi Kohno, and David Molnar. 2015. {SURROGATES}: Enabling {Near-Real-Time} dynamic analyses of embedded systems. In *9th USENIX Workshop on Offensive Technologies (WOOT 15)*.
- [83] Yang Lan, Di Jin, Zhun Wang, Wende Tan, Zheyu Ma, and Chao Zhang. 2023. Thunderkaller: Profiling and Improving the Performance of Syzkaller. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1567–1578. doi: 10.1109/ASE56229.2023.00124
- [84] Gwangmu Lee, Woohul Shim, and Byoungyoung Lee. 2021. Constraint-guided Directed Greybox Fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 3559–3576. <https://www.usenix.org/conference/usenixsecurity21/presentation/lee-gwangmu>
- [85] Gwangmu Lee, Duo Xu, Solmaz Salimi, Byoungyoung Lee, and Mathias Payer. 2024. SyzRisk: A Change-Pattern-Based Continuous Kernel Regression Fuzzer. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (Singapore, Singapore) (*ASIA CCS '24*). Association for Computing Machinery, New York, NY, USA, 1480–1494. doi: 10.1145/3634737.3637642
- [86] Philip Levis. 2012. Experiences from a Decade of TinyOS Development. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. USENIX Association, Hollywood, CA, 207–220. <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/levis>
- [87] Hongyu Li, Liwei Guo, Yexuan Yang, Shangguang Wang, and Mengwei Xu. 2024. An Empirical Study of Rust-for-Linux: The Success, Dissatisfaction, and Compromise. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 425–443. <https://www.usenix.org/conference/atc24/presentation/li-hongyu>
- [88] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 111 (apr 2024), 26 pages. doi: 10.1145/3649828
- [89] Wenqiang Li, Le Guan, Jingqiang Lin, Jiameng Shi, and Fengjun Li. 2021. From library portability to para-rehosting: Natively executing microcontroller software on commodity hardware. *arXiv preprint arXiv:2107.12867* (2021).
- [90] Wenqiang Li, Jiameng Shi, Fengjun Li, Jingqiang Lin, Wei Wang, and Le Guan. 2022. μ AFL: Non-Intrusive Feedback-Driven Fuzzing for Microcontroller Firmware. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) (*ICSE '22*). Association for Computing Machinery, New York, NY, USA, 1–12. doi: 10.1145/3510003.3510208
- [91] Xingwei Li, Yan Kang, Chenggang Wu, Danjun Liu, Jiming Wang, Yue Sun, Zehui Wu, Yunchao Wang, Rongkuan Ma, and Qiang Wei. 2025. Yesterday Once More: Facilitating Linux Kernel Bug Reproduction via Reverse Fuzzing. *IEEE Transactions on Information Forensics and Security* (2025), 1–1. doi: 10.1109/TIFS.2025.3562704
- [92] Yuwei Li, Yuan Chen, Shouling Ji, Xuhong Zhang, Guanglu Yan, Alex X. Liu, Chunming Wu, Zulie Pan, and Peng Lin. 2024. G-Fuzz: A Directed Fuzzing Framework for gVisor. *IEEE Transactions on Dependable and Secure Computing* 21, 1 (2024), 168–185. doi: 10.1109/TDSC.2023.3244825
- [93] Zhaofeng Li, Vikram Narayanan, Xiangdong Chen, Jerry Zhang, and Anton Burtsev. 2024. Rust for Linux: Understanding the Security Impact of Rust in the Linux Kernel. In *2024 Annual Computer Security Applications Conference (ACSAC)*. 548–562. doi: 10.1109/ACSAC63791.2024.00054
- [94] Zhenpeng Lin, Yueqi Chen, Yuhang Wu, Dongliang Mu, Chensheng Yu, Xinyu Xing, and Kang Li. 2022. GREBE: Unveiling Exploitation Potential for Linux Kernel Bugs. In *2022 IEEE Symposium on Security and Privacy (SP)*. 2078–2095. doi: 10.1109/SP46214.2022.9833683
- [95] Linux. 2024. Extensible Scheduler Class. <https://www.kernel.org/doc/html/next/scheduler/sched-ext.html>
- [96] Baozheng Liu, Chao Zhang, Guang Gong, Yishun Zeng, Haifeng Ruan, and Jianwei Zhuge. 2020. FANS: Fuzzing Android Native System Services via Automated Interface Analysis. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 307–323. <https://www.usenix.org/conference/usenixsecurity20/presentation/liu>
- [97] Dinghao Liu, Shouling Ji, Kangjie Lu, and Qinming He. 2024. Improving Indirect-Call Analysis in LLVM with Type and Data-Flow Co-Analysis. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 5895–5912. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-dinghao-improving>
- [98] Jianzhong Liu, Yuheng Shen, Yiru Xu, and Yu Jiang. 2024. Leveraging Binary Coverage for Effective Generation Guidance in Kernel Fuzzing. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) (*CCS '24*). Association for Computing Machinery, New York, NY, USA, 3763–3777. doi: 10.1145/3658644.3690232
- [99] Jianzhong Liu, Yuheng Shen, Yiru Xu, Hao Sun, and Yu Jiang. 2023. Horus: Accelerating Kernel Fuzzing through Efficient Host-VM Memory Access Procedures. *ACM Trans. Softw. Eng. Methodol.* 33, 1, Article 11 (Nov. 2023), 25 pages. doi: 10.1145/3611665
- [100] Peiyu Liu, Shouling Ji, Xuhong Zhang, Qinming Dai, Kangjie Lu, Lirong Fu, Wenzhi Chen, Peng Cheng, Wenhui Wang, and Raheem Beyah. 2021. IFIZZ: Deep-State and Efficient Fault-Scenario Generation to Test IoT Firmware. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 805–816. doi: 10.1109/ASE51524.2021.9678785
- [101] Qiang Liu, Cen Zhang, Lin Ma, Muhui Jiang, Yajin Zhou, Lei Wu, Wenbo Shen, Xiapu Luo, Yang Liu, and Kui Ren. 2021. FirmGuide: Boosting the Capability of Rehosting Embedded Linux Kernels through Model-Guided Kernel Execution. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 792–804. doi: 10.1109/ASE51524.2021.9678653

- [102] Chenyang Lyu, Shouling Ji, Chao Zhang, Yuwei Li, Wei-Han Lee, Yu Song, and Raheem Beyah. 2019. MOPT: Optimized Mutation Scheduling for Fuzzers. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 1949–1966. <https://www.usenix.org/conference/usenixsecurity19/presentation/lyu>
- [103] Tao Lyu, Liyi Zhang, Zhiyao Feng, Yueyang Pan, Yujie Ren, Meng Xu, Mathias Payer, and Sanidhya Kashyap. 2024. Monarch: A Fuzzing Framework for Distributed File Systems. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 529–543. <https://www.usenix.org/conference/atc24/presentation/lyu>
- [104] Zheyu Ma, Bodong Zhao, Letu Ren, Zheming Li, Siqi Ma, Xiapu Luo, and Chao Zhang. 2022. PrIntFuzz: Fuzzing Linux Drivers via Automated Virtual Device Simulation. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (Virtual, South Korea) (ISSTA 2022)*. Association for Computing Machinery, New York, NY, USA, 404–416. doi: 10.1145/3533767.3534226
- [105] Dominik Maier, Benedikt Radtke, and Bastian Harren. 2019. Unicornfuzz: On the Viability of Emulation for Kernel-space Fuzzing. In *13th USENIX Workshop on Offensive Technologies (WOOT 19)*. USENIX Association, Santa Clara, CA. <https://www.usenix.org/conference/woot19/presentation/maier>
- [106] Dominik Maier and Fabian Toepfer. 2021. BSOD: Binary-Only Scalable Fuzzing Of Device Drivers. In *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses (San Sebastian, Spain) (RAID '21)*. Association for Computing Machinery, New York, NY, USA, 48–61. doi: 10.1145/3471621.3471863
- [107] Valentin J.M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. 2021. The Art, Science, and Engineering of Fuzzing: A Survey. *IEEE Transactions on Software Engineering* 47, 11 (2021), 2312–2331. doi: 10.1109/TSE.2019.2946563
- [108] Alejandro Mera, Bo Feng, Long Lu, and Engin Kirda. 2021. DICE: Automatic emulation of dma input channels for dynamic firmware analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*. 1938–1954.
- [109] Alejandro Mera, Changming Liu, Ruimin Sun, Engin Kirda, and Long Lu. 2024. SHiFT: Semi-hosted Fuzz Testing for Embedded Applications. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA. <https://www.usenix.org/conference/usenixsecurity24/presentation/wu-jianliang>
- [110] Jonathan Metzman, László Szekeres, Laurent Maurice Romain Simon, Read Trevelin Sprabery, and Abhishek Arya. 2021. FuzzBench: An Open Fuzzer Benchmarking Platform and Service. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. New York, NY, USA.
- [111] Dongliang Mu, Yuhang Wu, Yueqi Chen, Zhenpeng Lin, Chensheng Yu, Xinyu Xing, and Gang Wang. 2022. An In-depth Analysis of Duplicated Linux Kernel Bug Reports. In *29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24-28, 2022*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/auto-draft-246/>
- [112] Marius Muench, Jan Stijohann, Frank Kargl, Aurélien Francillon, and Davide Balzarotti. 2024. What You Corrupt Is Not What You Crash: Challenges in Fuzzing Embedded Devices. In *Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18 - 26, 2024*. The Internet Society.
- [113] Stefan Nagy, Anh Nguyen-Tuong, Jason D. Hiser, Jack W. Davidson, and Matthew Hicks. 2021. Breaking Through Binaries: Compiler-quality Instrumentation for Better Binary-only Fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 1683–1700. <https://www.usenix.org/conference/usenixsecurity21/presentation/nagy>
- [114] Roberto Natella and Van-Thuan Pham. 2021. ProFuzzBench: a benchmark for stateful protocol fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (Virtual, Denmark) (ISSTA 2021)*. Association for Computing Machinery, New York, NY, USA, 662–665. doi: 10.1145/3460319.3469077
- [115] Aleksandr Nogikh. 2023. Syzbot: 7 years of continuous kernel fuzzing. In *Linux Plumbers Conference 2023*. <https://lpc.events/event/17/contributions/1521/>
- [116] Chandandeep Singh Pabla. 2009. Completely fair scheduler. *Linux Journal* 2009, 184 (2009), 4.
- [117] Shankara Pailoor, Andrew Aday, and Suman Jana. 2018. MoonShine: Optimizing OS Fuzzer Seed Selection with Trace Distillation. In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 729–743. <https://www.usenix.org/conference/usenixsecurity18/presentation/pailoor>
- [118] Jianfeng Pan, Guanglu Yan, and Xiaocao Fan. 2017. Digtool: A {virtualization-based} framework for detecting kernel vulnerabilities. In *26th USENIX Security Symposium (USENIX Security 17)*. 149–165.
- [119] Pallavi Pandey, Anupam Sarkar, and Ansuman Banerjee. 2019. Triforce QNX Syscall Fuzzer. In *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 59–60. doi: 10.1109/ISSREW.2019.00043
- [120] Hui Peng and Mathias Payer. 2020. USBFuzz: A Framework for Fuzzing USB Drivers by Device Emulation. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2559–2575. <https://www.usenix.org/conference/usenixsecurity20/presentation/peng>
- [121] Ivan Pustogarov, Qian Wu, and David Lie. 2020. Ex-vivo dynamic analysis framework for Android device drivers. In *2020 IEEE Symposium on Security and Privacy (SP)*. 1088–1105. doi: 10.1109/SP40000.2020.00094
- [122] Wang Qinying, Chang Boyu, Ji Shouling, Tian Yuan, Zhang Xuhong, Zhao Binbin, Pan Gaoning, Lyu Chenyang, Payer Mathias, Wang Wenhui, and Beyah Reheem. 2024. SyzTrust: State-aware Fuzzing on Trusted OS Designed for IoT Devices. In *2024 IEEE Symposium on*

- Security and Privacy (SP).*
- [123] Matthew J Renzelmann, Asim Kadav, and Michael M Swift. 2012. {SymDrive}: Testing Drivers without Devices. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. 279–292.
 - [124] Gabriel Ryan, Abhishek Shah, Dongdong She, and Suman Jana. 2023. Precise Detection of Kernel Data Races with Probabilistic Lockset Analysis. In *2023 IEEE Symposium on Security and Privacy (SP)*. 2086–2103. doi: 10.1109/SP46215.2023.10179366
 - [125] Tobias Scharnowski, Simon Wörner, Felix Buchmann, Moritz Schloegel Nils Bars, and Thorsten. 2023. Hoedur: Embedded Firmware Fuzzing using Multi-Stream Inputs. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 2885–2902. <https://www.usenix.org/conference/usenixsecurity23/presentation/tay>
 - [126] Nico Schiller, Merlin Chlosta, Moritz Schloegel, Nils Bars, Thorsten Eisenhofer, Tobias Scharnowski, Felix Domke, Lea Schönherr, and Thorsten Holz. 2023. Drone Security and the Mysterious Case of DJI's DroneID. In *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/drone-security-and-the-mysterious-case-of-djis-droneid/>
 - [127] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, and T. Holz. 2024. SoK: Prudent Evaluation Practices for Fuzzing. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 140–140. doi: 10.1109/SP54263.2024.00137
 - [128] Sergej Schumilo, Cornelius Aschermann, Ali Abbasi, Simon Wörner, and Thorsten Holz. 2021. Nyx: Greybox Hypervisor Fuzzing using Fast Snapshots and Affine Types. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2597–2614. <https://www.usenix.org/conference/usenixsecurity21/presentation/schumilo>
 - [129] Sergej Schumilo, Cornelius Aschermann, Robert Gawlik, Sebastian Schinzel, and Thorsten Holz. 2017. kAFL: Hardware-Assisted Feedback Fuzzing for OS Kernels. In *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, Vancouver, BC, 167–182. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/schumilo>
 - [130] Sergej Schumilo, Ralf Spennberg, and Hendrik Schwartke. 2014. Don't trust your USB! How to find bugs in USB device drivers. *Blackhat Europe* (2014).
 - [131] Yuheng Shen, Jianzhong Liu, Yiru Xu, Hao Sun, Mingzhe Wang, Nan Guan, Heyuan Shi, and Yu Jiang. 2024. Enhancing ROS System Fuzzing through Callback Tracing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (Vienna, Austria) (ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA.
 - [132] Yuheng Shen, Hao Sun, Yu Jiang, Heyuan Shi, Yixiao Yang, and Wanli Chang. 2021. Rtkaller: State-aware Task Generation for RTOS Fuzzing. *ACM Trans. Embed. Comput. Syst.* 20, 5s, Article 83 (Sept. 2021), 22 pages. doi: 10.1145/3477014
 - [133] Yuheng Shen, Yiru Xu, Hao Sun, Jianzhong Liu, Zichen Xu, Aiguo Cui, Heyuan Shi, and Yu Jiang. 2022. Tardis: Coverage-Guided Embedded Operating System Fuzzing. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4563–4574. doi: 10.1109/TCAD.2022.3198910
 - [134] Zekun Shen, Ritik Roongta, and Brendan Dolan-Gavitt. 2022. Drifuzz: Harvesting Bugs in Device Drivers from Golden Seeds. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 1275–1290. <https://www.usenix.org/conference/usenixsecurity22/presentation/shen-zekun>
 - [135] Heyuan Shi, Shijun Chen, Runzhe Wang, Yuhan Chen, Weibo Zhang, Qiang Zhang, Yuheng Shen, Xiaohai Shi, Chao Hu, and Yu Jiang. 2024. Industry Practice of Directed Kernel Fuzzing for Open-source Linux Distribution. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 2159–2169. <https://doi.org/10.1145/3691620.3695278>
 - [136] Heyuan Shi, Runzhe Wang, Ying Fu, Mingzhe Wang, Xiaohai Shi, Xun Jiao, Houbing Song, Yu Jiang, and Jianguang Sun. 2019. Industry practice of coverage-guided enterprise Linux kernel fuzzing. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Tallinn, Estonia) (ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 986–995. doi: 10.1145/3338906.3340460
 - [137] Jiameng Shi, Wenqiang Li, Wenwen Wang, and Le Guan. 2024. Facilitating Non-Intrusive In-Vivo Firmware Testing with Stateless Instrumentation. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society.
 - [138] SimonKagstrom. 2010. KCOV: code coverage for fuzzing. <https://docs.kernel.org/dev-tools/kcov.html>.
 - [139] Dokyung Song, Felicitas Hetzelt, Dipanjan Das, Chad Spensky, Yeoul Na, Stijn Volckaert, Giovanni Vigna, Christopher Kruegel, Jean-Pierre Seifert, and Michael Franz. 2019. PeriScope: An Effective Probing and Fuzzing Framework for the Hardware-OS Boundary. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/periscope-an-effective-probing-and-fuzzing-framework-for-the-hardware-os-boundary/>
 - [140] Dokyung Song, Felicitas Hetzelt, Jonghwan Kim, Brent ByungHoon Kang, Jean-Pierre Seifert, and Michael Franz. 2020. Agamotto: Accelerating Kernel Driver Fuzzing with Lightweight Virtual Machine Checkpoints. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2541–2557. <https://www.usenix.org/conference/usenixsecurity20/presentation/song>

- [141] Hao Sun, Yuheng Shen, Jianzhong Liu, Yiru Xu, and Yu Jiang. 2022. KSG: Augmenting Kernel Fuzzing with System Call Specification Generation. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 351–366. <https://www.usenix.org/conference/atc22/presentation/sun>
- [142] Hao Sun, Yuheng Shen, Cong Wang, Jianzhong Liu, Yu Jiang, Ting Chen, and Aiguo Cui. 2021. HEALER: Relation Learning Guided Kernel Fuzzing. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (Virtual Event, Germany) (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 344–358. doi: 10.1145/3477132.3483547
- [143] Hao Sun and Zhendong Su. 2024. Validating the eBPF Verifier via State Embedding. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 615–628. <https://www.usenix.org/conference/osdi24/presentation/sun-hao>
- [144] Hao Sun, Yiru Xu, Jianzhong Liu, Yuheng Shen, Nan Guan, and Yu Jiang. 2024. Finding Correctness Bugs in eBPF Verifier with Structured and Sanitized Program. In *Proceedings of the Nineteenth European Conference on Computer Systems (Athens, Greece) (EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 689–703. doi: 10.1145/3627703.3629562
- [145] Yue Sun, Yan Kang, Chenggang Wu, Kangjie Lu, Jiming Wang, Xingwei Li, Yuhao Hu, Jikai Ren, Yuanming Lai, Mengyao Xie, and Zhe Wang. 2025. SyzParam: Introducing Runtime Parameters into Kernel Driver Fuzzing. arXiv:2501.10002 [cs.CR] <https://arxiv.org/abs/2501.10002>
- [146] Matthias Hollick Sönke Huster and Jiska Classen. 2024. To Boldly Go Where No Fuzzer Has Gone Before: Finding Bugs in Linux' Wireless Stacks through VirtIO Devices. In *2024 IEEE Symposium on Security and Privacy (SP)*.
- [147] Seyed Mohammadjavad Seyed Talebi, Hamid Tavakoli, Hang Zhang, Zheng Zhang, Ardalan Amiri Sani, and Zhiyun Qian. 2018. Charm: Facilitating dynamic analysis of device drivers of mobile systems. In *27th USENIX Security Symposium (USENIX Security 18)*. 291–307.
- [148] Xin Tan, Yuan Zhang, Jiadong Lu, Xin Xiong, Zhuang Liu, and Min Yang. 2023. SyzDirect: Directed Greybox Fuzzing for Linux Kernel. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (Copenhagen, Denmark) (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 1630–1644. doi: 10.1145/3576915.3623146
- [149] Hui Jun Tay, Kyle Zeng, Jayakrishna Menon Vadayath, Arvind S Raj, Audrey Dutcher, Tejesh Reddy, Wil Gibbs, Zion Leonahenahe Basque, Fangzhou Dong, Zack Smith, Adam Doupé, Tiffany Bao, Yan Shoshitaishvili, and Ruoyu Wang. 2023. Greenhouse: Single-Service Rehosting of Linux-Based Firmware Binaries in User-Space Emulation. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5791–5808. <https://www.usenix.org/conference/usenixsecurity23/presentation/tay>
- [150] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2025. Fixing Large Language Models' Specification Misunderstanding for Better Code Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1514–1526. doi: 10.1109/ICSE55347.2025.00108
- [151] Linus Torvalds and the Linux Kernel Community. 1991. Linux Kernel. <https://www.kernel.org>.
- [152] Dmitry Vyukov. [n.d.]. syzbot. <https://syzkaller.appspot.com/upstream>.
- [153] Brian Walters. 1999. VMware virtual platform. *Linux journal* 1999, 63es (1999), 6–es.
- [154] Daimeng Wang, Zheng Zhang, Hang Zhang, Zhiyun Qian, Srikanth V. Krishnamurthy, and Nael Abu-Ghazaleh. 2021. SyzVegas: Beating Kernel Fuzzing Odds with Reinforcement Learning. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2741–2758. <https://www.usenix.org/conference/usenixsecurity21/presentation/wang-daimeng>
- [155] Jinghan Wang, Yue Duan, Wei Song, Heng Yin, and Chengyu Song. 2019. Be Sensitive and Collaborative: Analyzing Impact of Coverage Metrics in Greybox Fuzzing. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*. USENIX Association, Chaoyang District, Beijing, 1–15. <https://www.usenix.org/conference/raid2019/presentation/wang>
- [156] Jinghan Wang, Chengyu Song, and Heng Yin. 2021. Reinforcement Learning-based Hierarchical Seed Scheduling for Greybox Fuzzing. In *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21-25, 2021*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/reinforcement-learning-based-hierarchical-seed-scheduling-for-greybox-fuzzing/>
- [157] Pengfei Wang, Xu Zhou, Kai Lu, Tai Yue, and Yingying Liu. 2020. Sok: The progress, challenges, and perspectives of directed greybox fuzzing. *Challenges, and Perspectives of Directed Greybox Fuzzing* (2020).
- [158] Chen Weiteng, Hao Yu, Zhang Zheng, Zou Xiaochen, Kirat Dhilung, Mishra Shachee, Schales Douglas, Jang Jiyong, and Qian Zhiyun. 2024. SyzGen++: Dependency Inference for Augmenting Kernel Driver Fuzzing. In *2024 IEEE Symposium on Security and Privacy (SP)*.
- [159] Yuhang Wu, Zhenpeng Lin, Yueqi Chen, Dang K Le, Dongliang Mu, and Xinyu Xing. 2023. Mitigating Security Risks in Linux with KLAUS: A Method for Evaluating Patch Correctness. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 4247–4264. <https://www.usenix.org/conference/usenixsecurity23/presentation/wu-yuhang>
- [160] Yilun Wu, Tong Zhang, Changhee Jung, and Dongyoon Lee. 2023. DevFuzz: Automatic Device Model-Guided Device Driver Fuzzing. In *2023 IEEE Symposium on Security and Privacy (SP)*. 3246–3261. doi: 10.1109/SP46215.2023.10179293
- [161] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 126, 13 pages. doi: 10.1145/3597503.3639121
- [162] Hanxiang Xu, Wei Ma, Ting Zhou, Yanjie Zhao, Kai Chen, Qiang Hu, Yang Liu, and Haoyu Wang. 2025. CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph. In *2025 IEEE/ACM 47th International Conference on Software Engineering*:

- Companion Proceedings (ICSE-Companion)*. 243–254. doi: 10.1109/ICSE-Companion66252.2025.00079
- [163] Jiacheng Xu, Dylan Wolff, Xing Yi Han, Jialin Li, and Abhik Roychoudhury. 2025. Concurrency Testing in the Linux Kernel via eBPF. arXiv:2504.21394 [cs.OS]. <https://arxiv.org/abs/2504.21394>
 - [164] Jiacheng Xu, Xuhong Zhang, Shouling Ji, Yuan Tian, Binbin Zhao, Qinying Wang, Peng Cheng, and Jiming Chen. 2024. MOCK: Optimizing Kernel Fuzzing Mutation with Context-aware Dependency. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society.
 - [165] Meng Xu, Sanidhya Kashyap, Hanqing Zhao, and Taesoo Kim. 2020. Krace: Data Race Fuzzing for Kernel File Systems. In *2020 IEEE Symposium on Security and Privacy (SP)*. 1643–1660. doi: 10.1109/SP40000.2020.00078
 - [166] Qingxiao Xu and Jeff Huang. 2025. Optimizing Type Migration for LLM-Based C-to-Rust Translation: A Data Flow Graph Approach. In *Proceedings of the 14th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis (Seoul, Republic of Korea) (SOAP '25)*. Association for Computing Machinery, New York, NY, USA, 8–14. doi: 10.1145/3735544.3735582
 - [167] Wen Xu, Hyungon Moon, Sanidhya Kashyap, Po-Ning Tseng, and Taesoo Kim. 2019. Fuzzing File Systems via Two-Dimensional Input Space Exploration. In *2019 IEEE Symposium on Security and Privacy (SP)*. 818–834. doi: 10.1109/SP.2019.00035
 - [168] Chenyuan Yang, Zijie Zhao, and Lingming Zhang. 2025. KernelGPT: Enhanced Kernel Fuzzing via Large Language Models. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 560–573. doi: 10.1145/3676641.3716022
 - [169] Necip Fazıl Yildiran, Jeho Oh, Julia Lawall, and Paul Gazzillo. 2024. Maximizing Patch Coverage for Testing of Highly-Configurable Software without Exploding Build Times. *Proc. ACM Softw. Eng.* 1, FSE, Article 20 (July 2024), 23 pages. doi: 10.1145/3643746
 - [170] Tingting Yin, Zicong Gao, Zhenghang Xiao, Zheyu Ma, Min Zheng, and Chao Zhang. 2023. KextFuzz: Fuzzing macOS Kernel EXTensions on Apple Silicon via Exploiting Mitigations. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5039–5054. <https://www.usenix.org/conference/usenixsecurity23/presentation/yin>
 - [171] Xu Yiru, Sun Hao, Liu Jianzhong, Shen Yuheng, and Jiang Yu. 2024. SATURN: Host-Gadget Synergistic USB Driver Fuzzing. In *2024 IEEE Symposium on Security and Privacy (SP)*.
 - [172] Wei You, Peiyuan Zong, Kai Chen, XiaoFeng Wang, Xiaojing Liao, Pan Bian, and Bin Liang. 2017. SemFuzz: Semantics-based Automatic Generation of Proof-of-Concept Exploits. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 2139–2154. doi: 10.1145/3133956.3134085
 - [173] Hangchen Yu, Arthur Michener Peters, Amogh Akshintala, and Christopher J. Rossbach. 2020. AvA: Accelerated Virtualization of Accelerators. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 807–825. doi: 10.1145/3373376.3378466
 - [174] Ming Yuan, Bodong Zhao, Penghui Li, Jiashuo Liang, Xinhui Han, Xiapu Luo, and Chao Zhang. 2023. DDRace: Finding Concurrency UAF Vulnerabilities in Linux Drivers with Directed Fuzzing. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*. Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association. <https://www.usenix.org/conference/usenixsecurity23/presentation/yuan-ming>
 - [175] Tai Yue, Pengfei Wang, Yong Tang, Enze Wang, Bo Yu, Kai Lu, and Xu Zhou. 2020. EcoFuzz: Adaptive Energy-Saving Greybox Fuzzing as a Variant of the Adversarial Multi-Armed Bandit. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2307–2324. <https://www.usenix.org/conference/usenixsecurity20/presentation/yue>
 - [176] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, and Taesoo Kim. 2018. QSYM : A Practical Concolic Execution Engine Tailored for Hybrid Fuzzing. In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 745–761. <https://www.usenix.org/conference/usenixsecurity18/presentation/yun>
 - [177] Jooboom Yun, Fayozbek Rustamov, Juhwan Kim, and Youngjoo Shin. 2022. Fuzzing of Embedded Systems: A Survey. *Comput. Surveys* 55, 7 (2022), 1–33.
 - [178] Michal Zalewski. 2013. American Fuzzy Lop: a security-oriented fuzzer. <https://lcamtuf.coredump.cx/afl/>.
 - [179] Gen Zhang, Pengfei Wang, Tai Yue, Xiangdong Kong, Shan Huang, Xu Zhou, and Kai Lu. 2022. MobFuzz: Adaptive Multi-objective Optimization in Gray-box Fuzzing. In *29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24-28, 2022*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/auto-draft-199/>
 - [180] Lei Zhang, Keke Lian, Haoyu Xiao, Zhibo Zhang, Peng Liu, Yuan Zhang, Min Yang, and Haixin Duan. 2022. Exploit the Last Straw That Breaks Android Systems. In *2022 IEEE Symposium on Security and Privacy (SP)*. 2230–2247. doi: 10.1109/SP46214.2022.9833563
 - [181] Zhiyu Zhang, Longxing Li, Ruigang Liang, and Kai Chen. 2025. Unlocking Low Frequency Syscalls in Kernel Fuzzing with Dependency-Based RAG. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA038 (June 2025), 23 pages. doi: 10.1145/3728913
 - [182] Zhuo Zhang, Wei You, Guanhong Tao, Yousra Aafer, Xuwei Liu, and Xiangyu Zhang. 2021. StochFuzz: Sound and Cost-effective Fuzzing of Stripped Binaries by Incremental and Stochastic Rewriting. In *2021 IEEE Symposium on Security and Privacy (SP)*. 659–676. doi: 10.1109/SP40001.2021.00109

- [183] Bodong Zhao, Zheming Li, Shisong Qin, Zheyu Ma, Ming Yuan, Wenyu Zhu, Zhihong Tian, and Chao Zhang. 2022. StateFuzz: System Call-Based State-Aware Linux Driver Fuzzing. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 3273–3289. <https://www.usenix.org/conference/usenixsecurity22/presentation/zhao-bodong>
- [184] Wenjia Zhao, Kangjie Lu, Qiushi Wu, and Yong Qi. 2022. Semantic-Informed Driver Fuzzing Without Both the Hardware Devices and the Emulators. In *29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24-28, 2022*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/auto-draft-248/>
- [185] Yaowen Zheng, Ali Davanian, Heng Yin, Chengyu Song, Hongsong Zhu, and Limin Sun. 2019. FIRM-AFL: High-Throughput Greybox Fuzzing of IoT Firmware via Augmented Process Emulation. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 1099–1114. <https://www.usenix.org/conference/usenixsecurity19/presentation/zheng>
- [186] Yaowen Zheng, Yuekang Li, Cen Zhang, Hongsong Zhu, Yang Liu, and Limin Sun. 2022. Efficient greybox fuzzing of applications in Linux-based IoT devices via enhanced user-mode emulation. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (Virtual, South Korea) (ISSTA 2022)*. Association for Computing Machinery, New York, NY, USA, 417–428. doi: 10.1145/3533767.3534414
- [187] Wei Zhou, Le Guan, Peng Liu, and Yuqing Zhang. 2021. Automatic Firmware Emulation through Invalidity-guided Knowledge Inference. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2007–2024. <https://www.usenix.org/conference/usenixsecurity21/presentation/zhou>
- [188] Zhenhao Zhou, Zhuochen Huang, Yike He, Chong Wang, Jiajun Wang, Yijian Wu, Xin Peng, and Yiling Lou. 2025. Benchmarking and Enhancing LLM Agents in Localizing Linux Kernel Bugs. arXiv:2505.19489 [cs.AI] <https://arxiv.org/abs/2505.19489>
- [189] Jiaxun Zhu, Minghao Lin, Tingting Yin, Zechao Cai, Yu Wang, Rui Chang, and Wenbo Shen. 2024. CrossFire: Fuzzing macOS Cross-XPU Memory on Apple Silicon. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (Salt Lake City, UT, USA) (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 3749–3762. doi: 10.1145/3658644.3690376
- [190] Xiaogang Zhu, Sheng Wen, Seyit Camtepe, and Yang Xiang. 2022. Fuzzing: a survey for roadmap. *ACM Computing Surveys (CSUR)* 54, 11s (2022), 1–36.
- [191] Xiaochen Zou, Yu Hao, Zheng Zhang, Juefei Pu, Weiteng Chen, and Zhiyun Qian. 2024. SyzBridge: Bridging the Gap in Exploitability Assessment of Linux Kernel Bugs in the Linux Ecosystem. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society.
- [192] Xiaochen Zou, Guoren Li, Weiteng Chen, Hang Zhang, and Zhiyun Qian. 2022. SyzScope: Revealing High-Risk Security Impacts of Fuzzer-Exposed Bugs in Linux kernel. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 3201–3217. <https://www.usenix.org/conference/usenixsecurity22/presentation/zou>